UNIVERSIDADE DE SÃO PAULO – USP

ESCOLA POLITÉCNICA

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Eric Oliveira Gomes

Simulating Spiking Neural Networks: An Application to Liquid State Machines

São Paulo

2025

Eric Oliveira Gomes

Simulating Spiking Neural Networks: An Application to Liquid State Machines

Original Version

Monograph presented to the Departamento de Engenharia de Computação e Sistemas Digitais da Escola Politécnica da Universidade de São Paulo to obtain the title of Computer Engineer

Area of Concentration: Computer Engineering

Supervisor: Prof. Dr. Antonio Vieira da Silva Neto

São Paulo

2025

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio convencional ou eletrônico, para fins de estudo e pesquisa, desde que citada a fonte.

Catalogação-na-publicação

Gomes, Eric
Simulating Spiking Neural Networks: An Application to Liquid State Machines / E. Gomes -- São Paulo, 2025.
74 p.

Trabalho de Formatura - Escola Politécnica da Universidade de São Paulo. Departamento de Engenharia de Computação e Sistemas Digitais.

1.Computação Bioinspirada 2.Inteligência Artificial 3.Redes Neurais 4.Simulação I.Universidade de São Paulo. Escola Politécnica. Departamento de Engenharia de Computação e Sistemas Digitais II.t.

The brain is wider than the sky.
– Emily Dickinson

Resumo

Este trabalho cobre o projeto, a implementação e a avaliação de um simulador de atividade neural focado em redes neurais pulsadas, assim como uma análise da sua aplicabilidade a tarefas de aprendizado de máquina no contexto de *liquid state machines*. O simulador foi desenvolvido em *C++*, visando um alto nível de eficiência computacional, juntamente com uma interface de programação de aplicações (API) de alto nível intuitiva em *Python*. As decisões de projeto foram orientadas por uma revisão bibliográfica abrangente sobre a modelagem de atividade neural, que também auxiliou na definição dos requisitos funcionais e não funcionais do projeto. Com base nos requisitos e diretrizes arquiteturais definidos, foram projetados e implementados um núcleo de simulação orientado a eventos e a sua interface correspondente.

A corretude e o desempenho do simulador foram avaliados através de experimentos comparativos com ferramentas de simulação consolidadas, com foco no comportamento dinâmico de redes neurais simuladas. Os resultados confirmam que o sistema reproduz dinâmicas neurais conhecidas de forma semelhante aos outros simuladores, ao mesmo tempo em que alcança um nível de desempenho competitivo, satisfazendo, assim, os objetivos iniciais do projeto.

Para avaliar a aplicabilidade do simulador a tarefas de aprendizado de máquina, ele foi utilizado para instanciar um modelo de *liquid state machine* biologicamente inspirado, posteriormente aplicado ao conjunto de dados de classificação de áudio *Spiking Heidelberg Digits*. A exatidão obtida é comparável ao estado da arte de modelos do tipo *liquid state machine*, demonstrando a eficácia da arquitetura proposta. A aplicação da mesma arquitetura ao *dataset* de classificação de imagens N-MNIST também gera resultados com bons níveis de exatidão, demonstrando desempenho competitivo mesmo abaixo do estado da arte.

Em conclusão, o projeto resultou em um simulador de redes neurais pulsadas sucinto e eficiente, validado tanto por análises dinâmicas comparativas como por aplicações em aprendizado de máquina. No entanto, permanecem diversas oportunidades para melhorias futuras, incluindo execução com múltiplas *threads*, suporte a mecanismos de plasticidade sináptica, integração de modelos neuronais adicionais e a exploração de novos estudos de caso com tarefas de aprendizado de máquina.

Palavras-chave: Computação Bioinspirada. Inteligência Artificial. Redes Neurais. Simulação.

Abstract

This work covers the design, implementation and evaluation of a neural activity simulator, focused on spiking neural networks, as well as an assessment of its applicability to machine learning tasks within the liquid state machine framework. The simulator was developed in *C++* for computational efficiency, alongside a user-friendly high-level *Python* application programming interface (API). Design choices were guided by an extensive review of the theoretical background underlying neural activity modeling, which also aided in defining the projects functional and non-functional requirements. Drawing on the defined requirements and architectural guidelines, an event-driven simulation kernel and its associated interface were designed and implemented.

The simulator's correctness and performance were assessed through comparative experiments with well-established simulation tools, focusing on the dynamical behavior of the simulated neural networks. The results confirm that the simulator reproduces known neural dynamics similarly to other simulators, while also achieving performance comparable to the current state of the art, thereby satisfying the initial design goals.

In order to evaluate the simulator's applicability to machine learning tasks, it was used to instantiate a biologically-inspired liquid state machine model, which was then applied to the Spiking Heidelberg Digits audio classification benchmark. The obtained accuracy is comparable to state-of-the-art liquid state machines models, demonstrating the effectiveness of the proposed architecture. Applying the same architecture to the N-MNIST image classification dataset also yields competitive accuracy, although it does not reach state-of-the-art performance.

Overall, the project resulted in a succinct and efficient spiking neural network simulator, validated both through comparative dynamical analyses and machine learning applications. Nevertheless, several avenues for future enhancements remain, including multithreaded execution, support for synaptic plasticity mechanisms, the integration of further neuron models, as further case studies on machine learning tasks.

Keywords: Bio-inspired Computing. Artificial Intelligence. Neural Networks. Simulation.

List of Figures

List of Tables

List of Abbreviations and Acronyms

AI	Artificial Intelligence
ANN	Artificial Neural Network
API	Application Programming Interface
CMOS	Complementary Metal-Oxide-Semiconductor
CPU	Central Processing Unit
DVS	Dynamic Vision Sensor
EE	Excitatory to Excitatory
EI	Excitatory to Inhibitory
FR	Functional Requirement
GPU	Graphics Processing Unit
IE	Inhibitory to Excitatory
II	Inhibitory to Inhibitory
L-BFGS	Limited-memory Broyden-Fletcher-Goldfarb-Shanno
LIF	Leaky Integrate-and-Fire
LSM	Liquid State Machine
NEST	Neural Simulation Technology
NFR	Non-Functional Requirement
RC	Reservoir Computing
SHD	Spiking Heidelberg Digits
SI	Synchronous Irregular
SIMD	Single Instruction, Multiple Data
SNN	Spiking Neural Network
SoA	Structure of Arrays

STDP	Spike-Timing-Dependent Plasticity
UML	Unified Modeling Language
WTA	Winner-Take-All

Contents

1 Introduction

The study of neural activity has historically been a major area of interest in science, encompassing fields that range from fundamental neuroscience to advanced applications in artificial intelligence (Yamazaki et al., 2022) and biomedical engineering (Fan; Markram, 2019). Understanding how neurons interact, process information, and generate emergent complex behaviors is essential not only for advancing knowledge but also for creating the foundations for developing innovative technologies in areas such as artificial intelligence (Yamazaki et al., 2022), robotics (Li et al., 2021), and brain-machine interfaces (Tonin; Millán, 2021).

In this context, the growing computational power attained in the last decades made computational simulation of neural activity an increasingly powerful tool (Fan; Markram, 2019). Computational models of individual neurons and biological neural networks allow for the reproduction and exploration of these systems under a wide variety of conditions, without the need for direct experimentation on biological organisms. Beyond reducing costs and enabling large-scale testing, simulators provide a controlled environment that facilitates the analysis of specific parameters and the study of complex phenomena. Moreover, such systems can be explored directly to study the information processing capacities these biological systems, allowing for the construction of practical application of such systems (Yamazaki et al., 2022).

A popular technology that uses biologically inspired neurons are spiking neural networks (SNNs) (Maass, 1997; Yamazaki et al., 2022). This paradigm represents an alternative to conventional artificial neural networks (ANNs) (Goodfellow; Bengio; Courville, 2016) which constitute the foundation for deep learning. SNNs differ from ANNs by processing information through discrete spike events rather than continuous activations, enabling them to naturally capture temporal dynamics and sparse representations. This event-driven computation can lead to significantly lower energy consumption, particularly when implemented on neuromorphic hardware (Chundi et al., 2021; D'Agostino et al., 2024), making SNNs a promising candidate for real-time and low-power applications such as robotics, edge computing, and sensory processing.

Established software tools for simulating neural activity include *Brian 2* (Stimberg; Brette; Goodman, 2019), *NEURON* (Hines; Carnevale, 1997), and *NEST* (Neural Simulation Technology) (Diesmann; Gewaltig, 2001), all of which are widely used and evaluated in this study. These implementations allow for the simulation of neural behavior at different levels of detail, with varying levels of computational performance. However, applying such frameworks is bound to have limited flexibility, not allowing changes to certain

aspects of the implementations, such as data types used for processing, discretizations or unconventional modeling. Thus, it is pertinent to develop a novel neural simulator aimed at providing full control over the computational procedures employed in this approach.

One of the main motivations for this project was the author's participation in a research project during a double-degree exchange program in France at CentraleSupélec. That research focused on hardware capable of emulating neural activity. Consequently, the present project extends this work by exploring the same concepts within a software paradigm. This software-based exploration is a key step in enabling rapid, low-cost prototyping and experimentation prior to hardware implementation.

In this context, this work proposes a novel simulation software seeking to reproduce the behavior of a hardware accelerator built for emulating neural activity. Due to a nondisclosure agreement upheld by the author, details on the computations and mathematical models used by this hardware architecture are not provided or applied here. Nevertheless, emulating the precise behavior of this device motivates the creation of a novel framework for simulating neurons and biological neural networks, allowing for low-level execution control.

1.1 Objective

Then main objective of this work is to design, implement, and explore a neural activity simulator for the solution of machine learning tasks. This work aims to build upon mathematical models capable of reproducing the fundamental behaviors of neurons and biological neural networks. To achieve this goal, the following steps are proposed:

- a review of theoretical models of biological neurons,
- a review of existing software approaches for emulating neural activity,
- the computational implementation of a modular and extensible simulator,
- the execution of experiments to evaluate simulation behavior,
- the execution of experiments using this simulator for machine learning tasks,
- and the analysis of results in light of the scientific literature.

A distinctive feature of the simulator is that it prioritizes the overall dynamics of information processing over precise biological accuracy, enabling faster experimentation with large input sets. Consequently, simple neuron models with few state variables are implemented, aiming to improve performance at the expense of behavioral accuracy. This strategy also aims to reduce memory usage. This design choice is made with the intent of

building artificial intelligence applications from simulated networks, since it is essential in this context to use a fast simulator for rapid training and inference.

A particular machine learning paradigm is explored: liquid state machines (LSMs) (Maass; Natschläger; Markram, 2002), a computational framework within the broader field of reservoir computing (Jaeger, 2001), in which a dynamical system, called a reservoir, processes time-varying inputs and builds rich representations by exploiting the system's dynamics. Here, a neural network instantiated on the developed neural simulator is to be used as a LSM. This paradigm is chosen since it allows the use of a network with fixed weights, which simplifies the training procedure, reduces the number of trainable parameters, and enables the exploration of the processing capabilities of bio-inspired network topologies.

1.2 Motivation

The main motivation behind building a neural activity simulator is the increasing relevance of energy efficiency and scalability in machine learning. Spiking neural networks offer an energy-efficient alternative, specially when executed on neuromorphic hardware (Chundi et al., 2021; D'Agostino et al., 2024). They also provide an effective framework for processing time-varying data, particularly in scenarios involving sparsely encoded inputs (Yamazaki et al., 2022).

The motivation for developing a novel neural activity simulator arises from the limitations of existing tools when applied to hardware-specific contexts. While simulators such as Brian 2, NEURON, and NEST (Stimberg; Brette; Goodman, 2019; Hines; Carnevale, 1997; Diesmann; Gewaltig, 2001) have become standards in the field, they are designed primarily for biological plausibility and general-purpose neuroscience research. As a result, their computational architectures often prevent direct adaptation to scenarios where low-level flexibility or hardware emulation are required. In this work, the goal is to build a simulator that can later be adapted to emulate a neuromorphic hardware implementation, which is not described here due to a nondisclosure agreement.

By addressing low-level computational aspects, the simulator is also designed to facilitate experimentation with simplified neuron models, which can be employed to run faster simulations. The primary motivation for reducing simulation time is to enable shorter training cycles and lower inference latency in the resulting networks. This, in turn, supports faster experimental iterations, thereby enhancing opportunities for the exploration and optimization of such systems.

Finally, the exploration of LSM computational framework seeks to provide a perspective of possible applications of the developed simulator. The proposed simulator therefore serves a dual role: not only as a tool for reproducing neuron and network

dynamics, but also as an experimental platform for building practical machine learning solutions. In sum, this project is motivated by the intersection of neuroscience-inspired computation and real-world AI applications.

1.3 Outline

Chapter 2 covers the theoretical foundations necessary for the development of the simulator described in this document. Chapter 3 comprises the methods adopted in this project, including the modeling choices for neural representations and the main decisions regarding the software implementation. Chapter 4 contains a description of the developed software, with emphasis on its main features. Chapter 5 covers the simulated experiments, including both the validation of the simulator's correctness and its application to selected machine learning tasks. Chapter 6 is devoted to discussing the obtained results by analyzing the strengths and limitations of the simulator and comparing its performance with existing neural activity simulators. Finally, Chapter 7 has concluding remarks and the outlines of potential directions for future work.

2 Theoretical Background

In this chapter, the theoretical background relevant to this work is introduced. Initially, an introduction to mathematical models of neurons, including the Hodgkin-Huxley and Integrate-and-Fire models, is provided. Next, the concept of Spiking Neural Networks (SNNs) and their relevance in machine learning is discussed. Finally, the principles of Reservoir Computing (RC) are presented.

2.1 Neural Models

One of the first models of neural behavior stems from the pioneering work of Louis Lapicque, who introduced the Leaky Leaky Integrate-and-Fire (LIF) neuron model in 1907 (Lapicque, 2007). Lapicque's goal was to provide a mathematical description of the neuron's excitability, linking the membrane potential dynamics to the generation of action potentials (spike-like voltage oscillations).

Lapicque's model quickly became a cornerstone of theoretical neuroscience, bridging the gap between biological observations and mathematical formalism. Despite its simplicity, it remained influential for decades and laid the foundation for more complex conductance-based models, such as the Hodgkin-Huxley model, which present a better biological precision.

The Hodgkin-Huxley model, introduced by Alan Hodgkin and Andrew Huxley (Hodgkin; Huxley, 1952), represents a major landmark in computational neuroscience, providing the first biophysically detailed mathematical description of how action potentials are generated and propagated in neurons. Based on experiments with the squid giant axon, the model considers the membrane as an electrical circuit in which voltage-gated sodium and potassium channels, together with a passive leak current, govern the dynamics of the membrane potential. By formulating a set of coupled nonlinear differential equations, Hodgkin and Huxley were able to capture both the initiation and the temporal profile of the action potential, in close agreement with experimental data. Unlike the simplified integrate-and-fire model, the Hodgkin-Huxley framework explicitly incorporates ion conductances and gating variables, thereby linking cellular electrophysiology with measurable biophysical parameters.

However, the simulation of large scale networks of Hodgkin-Huxley neurons remains unfeasible due to the computational cost of numerically solving its coupled differential equations. As a consequence, the LIF model and its variations are still widely used in computational neuroscience. The LIF model is most widely applied when biological accuracy

is not a priority, allowing for the simulation of large scale networks with a manageable computational cost. A variety of modern biological neuron models has been proposed, balancing different levels of biological accuracy against computational efficiency (Gerstner; Kistler, 2002).

A LIF neuron is modeled by Equation (2.1).

$$\tau_m \frac{dV(t)}{dt} = -\Big(V(t) - V_{\text{rest}}\Big) + R_m I(t) \tag{2.1}$$

In this equation, the variable V denotes the membrane potential, while I corresponds to the input current provided by presynaptic neurons, i.e., neurons transmitting signals to the neuron under consideration. The quantities τ_m, V_{rest}, and R_m are constants. These equations are analogous to those governing a simple RC circuit: the neuron is represented as a capacitor C that accumulates charge through the input current I, while discharging through a parallel resistor R_m. In this setting, the membrane time constant is given by $\tau_m = R_m C$. Furthermore, a potential V_{rest} introduced as a voltage source to circuit, accounting for the neuron's intrinsic tendency to return to a resting potential that is not necessarily zero.

In biological neurons, the membrane resistance R_m reflects how easily ions can leak across the membrane through ion channels, the membrane capacitance C represents the ability of the membrane's lipid bilayer to store charge, and V_{rest} is the voltage maintained by ion gradients when no synaptic input is present.

An analytical solution, described in Equation (2.2), can be obtained when $I(t) = 0$, given an initial potential V_0 at a time t_0.

$$V(t) = V_{\text{rest}} + \Big(V_0 - V_{\text{rest}}\Big) e^{\frac{t-t_0}{\tau_m}} \tag{2.2}$$

2.2 Spiking Neural Networks

Spiking Neural Networks (SNNs) are a class of neural networks that more closely mimic the behavior of biological neurons compared to traditional Artificial Neural Networks (ANNs), the predominant architecture currently used for deep learning (Goodfellow; Bengio; Courville, 2016). Unlike ANNs, in which neurons communicate using continuous activation values, neurons in SNNs communicate through discrete spikes. This design allows information to be encoded either in the timing of individual spikes or in the rate of spiking events. This event-driven mechanism introduces a temporal dimension to computation, which is absent in standard ANNs.

ANNs typically rely on simplified neuron models such as perceptrons (Rosenblatt, 1958) or neurons with other non-linear activations (Goodfellow; Bengio; Courville, 2016),

whose outputs are smooth, differentiable functions of the inputs. In contrast, SNNs employ biologically-inspired models like the Leaky Integrate-and-Fire (LIF) neuron or Hodgkin-Huxley models, which accumulate input over time and generate discrete spikes only when a threshold is reached. This difference not only introduces temporal dynamics but also allows SNNs to naturally handle time-dependent signals, making them well-suited for tasks such as audio processing or event-based sensor data processing.

From a computational perspective, ANNs perform continuous, dense computations at every step, typically relying on matrix multiplications for forward and backprogatation, usually being trained through gradient-based methods. SNNs, however, operate in an event-driven manner, that is, computations are triggered only when a spike event occurs, which can lead to significant energy savings, particularly when implemented on neuromorphic hardware which is designed under this event-driven premisse. However, training SNNs remains significantly more challenging due to the non-differentiable nature of spikes (Yamazaki et al., 2022). This fact motivated the creation of specialized techniques, such as surrogate gradient methods (Neftci; Mostafa; Zenke, 2019) or spike-timing-dependent plasticity (STDP) (Song; Miller; Abbott, 2000).

The surrogate gradient method enables gradient-based optimization in SNNs, in which direct backpropagation is hindered by the non-differentiable spiking nonlinearity. In order to circumvent the undefined derivative of the spike function, the backward pass uses a smooth, differentiable approximation to propagate error signals through the network, allowing for the gradient to be calculated (Neftci; Mostafa; Zenke, 2019).

In contrast, STDP is a biologically inspired learning rule that adjusts synaptic weights based on the precise timing of spikes between pairs of neurons. By exploiting temporal dependencies, neural circuits can learn correlations directly from spike timing. As a result, STDP enables unsupervised learning without relying on gradient-based optimization, providing a model of synaptic plasticity observed in the brain (Song; Miller; Abbott, 2000).

Conversion from traditional ANNs to rate-based SNNs is another approach used for network definition and training, allowing the reuse of pre-trained weights and biases. This strategy has been proposed as a way to improve energy efficiency at deployment time, leveraging the event-driven activity of SNNs to replace the heavy computations required by conventional ANNs. Nevertheless, this approach is not yet applicable to all ANN architectures (Yamazaki et al., 2022).

Overall, while ANNs remain dominant in most machine learning applications due to their simplicity and mature training methods, SNNs offer advantages in energy efficiency, temporal processing, and biological plausibility, representing a promising direction for new efficient computing paradigms.

2.3 Reservoir Computing

Reservoir computing (RC) is a computational paradigm that seeks to exploit the dynamics of recurrent systems to process temporal information, leveraging intrinsic properties such as memory and nonlinearity to construct rich features for downstream tasks (Lukoševičius; Jaeger, 2009). Figure 1 illustrates the main components of a reservoir computing system. An input layer feeds signals to the reservoir, which transforms the temporal input into a high-dimensional dynamic state, effectively creating a rich representation from the input's history. A separate readout layer, the only trainable part of the model, then maps these reservoir states to desired outputs, typically through simple, often linear, computations. The reservoir's intrinsic dynamics are leveraged to capture temporal patterns, making them suitable for a variety of tasks, such as speech recognition, time-series prediction, and real-time signal processing, while requiring minimal training of the network itself.

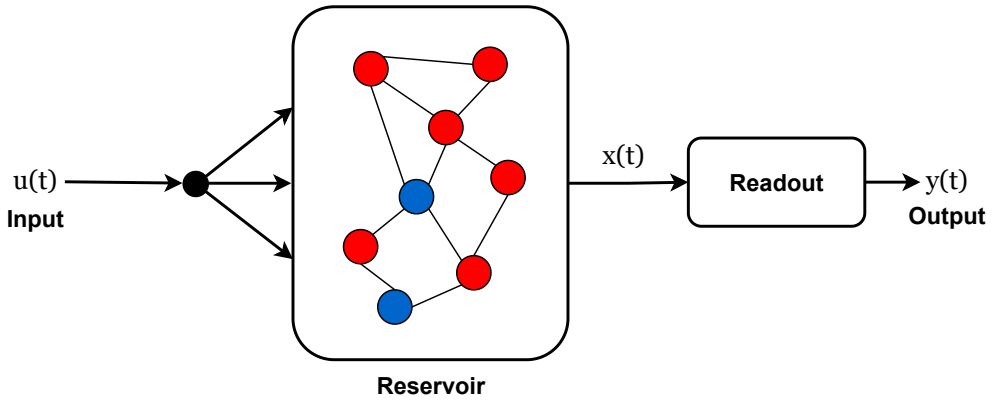


Figure 1 – Main components of a reservoir computer, showing the input layer that maps the external signal $\mathbf{u}(t)$ into the reservoir, the recurrent reservoir dynamics generating the high-dimensional state $\mathbf{x}(t)$, and the readout layer producing the output $\mathbf{y}(t)$.

Source: Adapted from Yan et al. (2024).

In this work, the Liquid State Machine (LSM) framework is studied. Historically, this framework has provided the theoretical foundations for constructing reservoirs from networks of spiking neurons (Maass; Natschläger; Markram, 2002), although it can also be applied to other physical systems. Within this approach, a reservoir is built from a recurrent SNN with randomly initialized weights. The dynamics of the system are thus determined by the network's topology, its synaptic weights and propagation delays, as well as the chosen neural model and its parameters.

An LSM can be formally described by a system of 2 equations (Maass; Natschläger; Markram, 2002):

$$x^M(t) = \left(L^M u\right)(t), \tag{2.3a}$$

$$y(t) = f^M\left(x^M(t)\right). \tag{2.3b}$$

Equation (2.3a) describes the evolution of the reservoir's state. x^M is a function of time representing the reservoir's state, while u is a function representing the reservoir input signals. Lastly, L^M is an operator that links these terms. Equation (2.3b) describes the generation of useful outputs from the reservoir's state. f^M represents the trainable readout function, and y, the output of interest. Note that multiple readout functions can be trained on a single reservoir state function, allowing multiple tasks to be performed in parallel.

The reservoir computing approach presents a few key advantages. Firstly, its training procedure is greatly simplified, as only the parameters of a readout function have to be trained. The reservoir itself is fixed and usually defined by a small set of hyperparameters, which may optionally be optimized. Hence, the training procedure is usually faster and less computationally expensive compared to training the full network (Lukoševičius; Jaeger, 2009). Secondly, due to its inherent flexibility, it can be used to construct networks from unconventional physical systems, even if they do not allow for gradient-based parameter optimization. This property has been explored to investigate a variety of systems that may offer lower energy consumption or faster processing speeds compared to conventional CMOS-based computers. Some examples of this are photonic (optic) (Sande; Brunner; Soriano, 2017; Larger et al., 2017; Antonik et al., 2019), memristor-based (Du et al., 2017; Zhong et al., 2022) and biological (Cai et al., 2023; Sumi et al., 2023) reservoirs.

2.4 Neuromorphic computing datasets

In neuromorphic computing research, it is common to make use of datasets which present data in an event-based format compatible with bio-inspired methods. These datasets provide information as spike events over time rather than as continuous values, which would otherwise require encoding into an event-based format. The Spiking Heidelberg Digits and N-MNIST datasets are used in the case studies presented in this work.

2.4.1 Spiking Heidelberg Digits

The Spiking Heidelberg Digits (SHD) dataset (Cramer et al., 2022) is a common benchmark dataset for SNNs. It consists of spike-encoded audio files corresponding to spoken digits from 0 to 9 in English and German. The spike encoding used by this dataset mimics the signals generated by the human cochlea in response to sound waves, providing a way to study how biological neural networks could process auditory stimuli.

This dataset contains a total of 10 420 recordings, with 8 156 used for training and the remaining 2 264 for testing. The sample labels 0 to 9 denote the spoken digits from 0 to 9 in English, whereas the labels from 10 to 19 correspond to the digits from 0 to 9 in German. This convention is followed in the results presented in further sections.

Figure 2 presents a sample of the spoken digit "one" from the SHD dataset as a raster plot, depicting spike events across the different input channels over time. Each dot in the raster plot represents a spike event from one of the input neurons (input channels). In the SHD task, each neuron is indirectly associated with a specific frequency band of the audio signal, firing when energy is present in that band (Cramer et al., 2022).

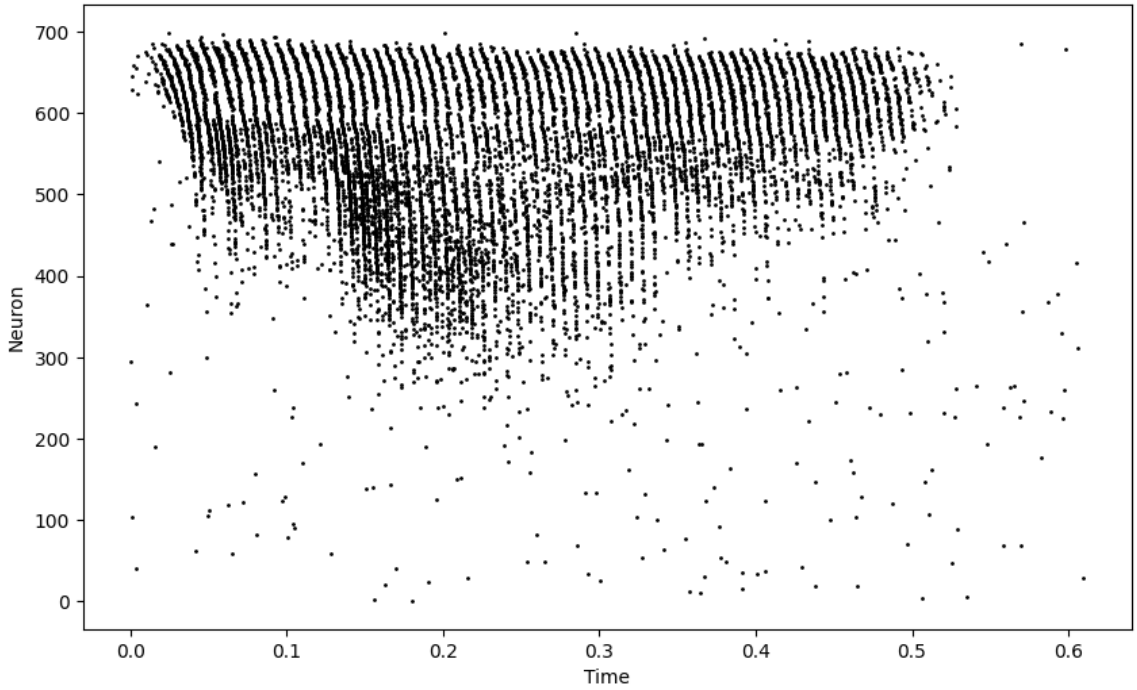


Figure 2 – Raster plot illustrating spike event pattern for a sample of the spoken digit "one" from the SHD training set (Cramer et al., 2022).

Table 1 summarizes the best performing models for the SHD dataset, also highlighting the best performing liquid state machines. The best performing models are SNNs with procedures for optimizing synaptic weights and delays (Sun et al., 2025; Schöne et al., 2024; Baronig et al., 2025; Hammouamri; Khalfaoui-Hassani; Masquelier, 2023), employing surrogate gradients to enable backpropagation through time (Neftci; Mostafa; Zenke, 2019). This approach enables gradient-based optimization, making its training procedure similar to that of usual ANNs.

A substantial performance gap is observed between the state-of-the-art models and the best-performing LSM. This is expected, as a finely optimized network should be capable of learning more efficient representations than those produced by the efficient random initialization typical of LSMs. Nevertheless, the simplicity and generality of the LSM approach make such solutions worth exploring, particularly for physical implementations which are incompatible with gradient-based updates (Sande; Brunner; Soriano, 2017).

Table 1 – Classification accuracies and parameter counts of the best-performing models on the SHD benchmark.

Model	**Test accuracy (%)**	**Trainable parameters**
Sun et al. (2025)	96.26 ± 0.08	200 000
Schöne et al. (2024)	95.90	400 000
Baronig et al. (2025)	95.81 ± 0.56	450 000
Hammouamri, Khalfaoui-Hassani and Masquelier (2023)	95.07 ± 0.24	200 000
	...	
Biswas et al. (2024)*	77.80	30 000
Krenzer and Bogdan (2025)*	65.80	–

* Liquid state machine approaches.

2.4.2 N-MNIST

The N-MNIST dataset (Orchard et al., 2015) is an event-based version of the well-known MNIST image classification dataset (Lecun et al., 1998), which consists of grayscale images of handwritten digits from 0 to 9. It was generated by recording the original MNIST images using a Dynamic Vision Sensor (DVS), a camera that detects only changes in brightness over time, capturing motion through events instead of full image frames. Due to this characteristic, a motorized pan-tilt unit was used during the recording process to produce sensor movements, allowing the DVS to produce streams of events corresponding to the original static images.

Similarly to the original MNIST dataset, N-MNIST contains a total of 70 000 samples, with 60 000 used for training and 10 000 for testing. Each sample consists of a 300 ms sequence of events, with each event mapped to a position in a 34×34 pixel grid and associated with a polarity value that encodes the direction of brightness change (either an increase or a decrease) at the corresponding spatial position. Figure 3 presents groups of events grouped over time intervals for a sample of the digits “0” from the N-MNIST dataset. Here, the two-dimensional structure of the original images is preserved, allowing for the visual identification of the digit represented by the events.

This format makes N-MNIST particularly suitable for evaluating spiking neural networks and neuromorphic systems in general, as the event-based nature of data makes it compatible with the event-based processing adopted by these systems. This enables direct use of data, without the need for arbitrary input encoding schemes.

The research that first introduced the N-MNIST dataset presents models for the classification task which may be adopted as a baseline for performance, achieving maximum accuracy of 83.44% (Orchard et al., 2015). In recent works, SNN methods akin to those employed on the SHD dataset have been adopted, notably in the SNN architecture proposed by Zhang et al. (2024), which attained a reported accuracy of 99.57%.

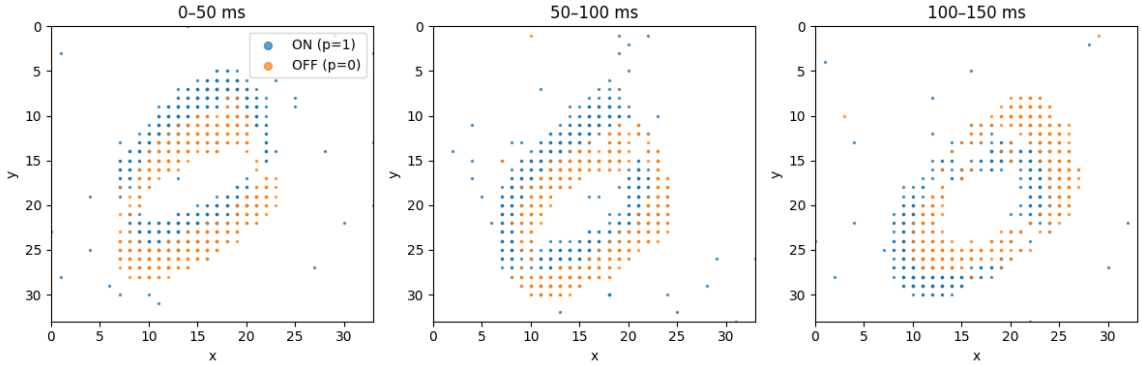


Figure 3 – Events grouped over time intervals for a sample of the digit "0" from the N-MNIST training set (Orchard et al., 2015), represented over the corresponding two-dimensional grid.

Similarly to the SHD dataset, LSM approaches have also been applied to N-MNIST, achieving lower accuracies compared to finely optimized SNNs. Biswas *et al.* report an accuracy of 98.1% for an LSM ensemble (Biswas et al., 2024), which represents the highest accuracy obtained with an LSM in the reviewed literature.

3 Materials and Methods

In this chapter, the materials and methods employed in this project are detailed. Firstly, the functional and non-functional requirements are defined, establishing a foundation for what should be developed. Taking these requirements into consideration, the technologies to be employed are defined, including both the hardware, which serves as the basis for evaluating the implementations, and the software used to develop the simulator. This chapter also outlines the methods employed throughout the project, including the techniques used to obtain numerical solutions for the neural models, the simulator architecture selected as the basis for the implementation, and the methods used to assess its correctness. Lastly, the work plan adopted for this project's execution is described.

3.1 Requirements

The objective of this section is to define the functional and non-functional requirements that guide the development of the proposed simulator. The functional requirements specify the essential capabilities this simulator must provide to users, that is, describing what the simulator is expected to do. The non-functional requirements outline the quality attributes and constraints under which the system shall operate. Together, these requirements provide the foundation for design and implementation choices taken throughout the project.

The functional requirements are listed in Table 2, whereas the non-functional requirements are provided in Table 3. The functional requirements are designed to support neural activity simulation while also establishing the key distinctions of this implementation compared to existing simulators, notably through the adoption of a purely event-driven approach. They also establish integration with *NumPy*, a *Python* library for array computations (Harris et al., 2020), which is compatible with both Brian 2 (Stimberg; Brette; Goodman, 2019) and NEST (Diesmann; Gewaltig, 2001). In contrast, the non-functional requirements are defined to ensure the simulator's effective use in machine learning tasks. State-of-the-art LSM methods often employ networks on the order of 1,000 neurons (Biswas et al., 2024; Krenzer; Bogdan, 2025), motivating support for networks of this scale with sparse connectivity. Additionally, the simulator must be capable of sub-second processing of inputs in these cases, ensuring its applicability in low-latency tasks. The project shall also be designed for extension, allowing for future development. Finally, the simulator must be user-accessible. Since user testing is not feasible within the scope of the project, this requirement will be preliminarily validated through the creation of an interface with a simple high-level interaction loop, minimizing its complexity.

Table 2 – Functional requirements for the simulator

ID	Requirement
FR1	**Neuron simulation:** The simulator shall implement mathematical models for neurons, reproducing behavior observed in biological counterparts.
FR2	**Network simulation:** The simulator shall allow the combination of neuron objects into networks and support their simulation under external stimuli.
FR3	**Event-driven computation:** The simulator shall process information through discrete events rather than continuous activations used in conventional ANNs.
FR4	**Experiment execution:** The simulator shall support experiments with access to spiking activity and network states throughout execution, enabling neural behavior studies and machine learning applications.
FR5	**Low-level control:** The simulator shall provide direct control over computational procedures, supporting the extension of existing numerical representations and processing strategies through the creation of subclasses.
FR6	**Accessible interface:** A high-level programmatic interface shall expose the simulator kernel, enabling the definition of simulations, experiment execution, and result retrieval. Simulation results shall be retrievable as *NumPy* arrays.

Table 3 – Non-functional requirements for the simulator

ID	Requirement
NFR1	**Performance:** The simulator shall be computationally efficient, achieving competitive performance. As an acceptance criterion, a liquid state machine of 1,000 neurons must simulate 1 second of biological time in under 1 second of elapsed time. Benchmark architectures similar to those used by Stimberg, Brette and Goodman (2019), who introduced Brian 2, will also be used for comparison, with the simulator expected to perform comparably to Brian 2 and NEST.
NFR2	**Scalability:** The simulator shall support networks of varying sizes. As a baseline, it must simulate both a single-neuron network and a 1,000-neuron network with 10% connection density (100,000 synapses).
NFR3	**Adaptability:** The codebase shall be designed for easy extension and modification of existing functionalities, using object-oriented programming to create generic classes from which multiple implementations can inherit.
NFR4	**Accessible interface:** The programmatic interface shall be intuitive, with the use of clear naming. It shall also be user-friendly, which should be validated with user testing. If user testing is not feasible, this requirement may be preliminarily validated through an analysis of the interface design.

3.2 Materials

In this section, the hardware and software resources used in this project are listed. This includes the computer and operating system used for development and testing, as well as the build tools, programming languages and libraries employed in the implementation.

3.2.1 Hardware and Operating System

The development and testing of the neural simulator was conducted on a personal computer running the Ubuntu 22.04.5 LTS Linux distribution, a stable long-term support version of Linux. The system was equipped with an Intel Core i7-10750H processor, featuring 6 cores and 12 threads with a base clock frequency of 2.6 GHz. The machine had 16 GB of DDR4-2666 RAM. All experiments were executed on the CPU; no GPU-based experiments were conducted.

3.2.2 Software

The *C++20* programming language was chosen for developing the simulator due to its ability to provide fine-grained control over low-level computations. This includes precise management of data types, memory allocation, and even processor instructions in some cases, such as the use of vector extensions. Additionally, being a compiled language, *C++* avoids the runtime overhead associated with interpreted languages, ensuring more efficient execution of performance-critical routines. For these reason, this language was also employed to build many of the popular neural simulators, such as *Brian 2* (Stimberg; Brette; Goodman, 2019) and *NEST* (Diesmann; Gewaltig, 2001).

Auxiliary software was employed to enable efficient development in the *C++* language. The *GNU C++ Compiler* (version 11.4.0) was used to compile the programs into executable machine code. To assist in the build process, the *GNU Make* (version 4.3) tool was utilized. On top of this, the *CMake* (version 3.22.1) build system facilitated better project organization and the inclusion of external libraries.

In addition to the simulator, an interface in the *Python* (version 3.10.12) language was created to provide an easier way to interact with the system. The *pybind11* library (version 3.0.1) was used to establish interoperability between *C++* and *Python*.

3.3 Methods

This section describes the methods employed in this project, including the adopted neuron and synapse models and the simulator architecture. These design choices provide the foundation for the implementation of the neural simulator.

3.3.1 Neuron and Synapse Models

In this work, the leaky integrate-and-fire (LIF) neuron model detailed in Chapter 2 was adopted. Several key advantages motivated this choice: its low computational cost for simulation, the availability of analytical solutions for certain input currents, and its widespread use within the spiking neural network (SNN) framework. The synapse model

employed here is current-based, chosen for its simplicity and computational efficiency. Hence, it is worth highlighting that a pragmatic approach to simulated neural activity was adopted in order to prioritize faster simulation, an essential requirement for information processing applications, over strict biological accuracy.

Another key simplification is that neuron spike events and synaptic transmissions are considered as instantaneous, that is, post-synaptic currents and potentials are modeled as Dirac delta functions ($\delta(x)$) multiplied by constant values ($C \cdot \delta(x), C \in \mathbb{R}$). The Dirac delta function has the following properties:

1. Zero everywhere except at zero, as detailed in Equation (3.1).

$$\delta(x) = 0 \quad \text{for all } x \neq 0 \tag{3.1}$$

2. Integral equals one, as described in Equation (3.2).

$$\int_{-\infty}^{\infty} \delta(x)\, dx = 1 \tag{3.2}$$

Even though these processes are instantaneous in this simulation architecture, a transmission delay can still be introduced between any two connected neurons. Consequently, each synaptic connection has two parameters: a propagation delay and a synaptic weight, the latter determining how much a passing spike depolarizes or hyperpolarizes the postsynaptic neuron's membrane.

3.3.2 Simulator Architecture

From an user's perspective, the simulator provides two interfaces, as shown in Figure 4. The first one is a simulation kernel library implemented in *C++*, which contains the neuron models and the core simulation routines. Extensions to the simulator's capabilities are primarily implemented at this level. This *C++* implementation is compiled to produce efficient executable routines.

However, for the end user, the *C++* interface can be difficult to use, as it does not easily integrate with most data analysis and machine learning tools. To address this, a transparent *Python* interface is provided, allowing for more straightforward control of the simulation environment. This interface enables users to define and interact with the neural network, including the specification of the simulation time frame, neuron parameters, network topology, and external spike inputs. It also supports the retrieval of spike event traces and the overall system state during a simulation. In addition, optional debug signals can be added to facilitate detailed analysis.

In order to simulate the precise timing of neural dynamics, an event-based simulation approach is adopted. This means that state variables are only updated when

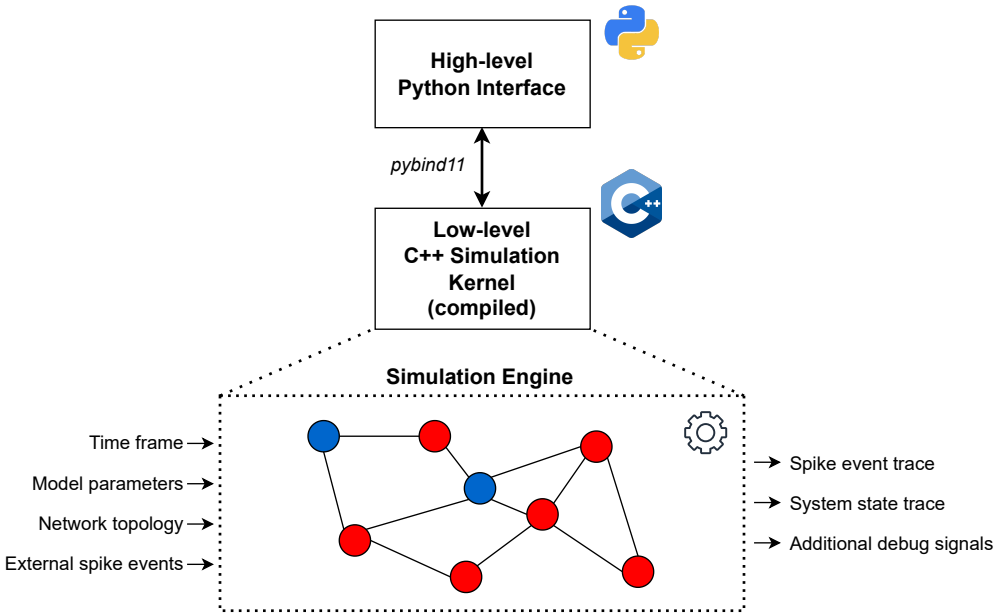


Figure 4 – Integration between the low-level *C++* simulation kernel and the high-level *Python* experimental interface.

a significant event happens, instead of updating the state continuously according to a certain time step. Adopting this paradigm also reduces computation when the network is sufficiently sparse. This approach is compared with time-driven simulations in Figure 5, where the periodic updates of the time-driven approach are contrasted with the less frequent updates triggered by presynaptic spikes in the event-driven method.

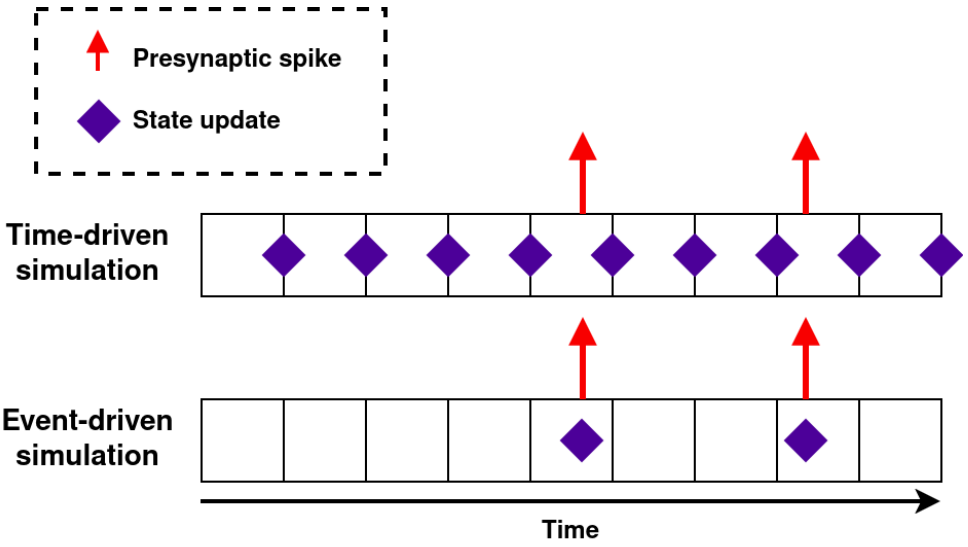


Figure 5 – Comparison between time-driven and event-driven neural activity simulations.

It is worth noting that for the LIF neuron model, a spike can only be generated after a spike happens in a presynaptic neuron, as it is the only situation in which the membrane potentials may increase. This property is essential for the event-driven approach to work. The existence of an analytical solution to the neuron model makes it possible to compute

the state evolution over any time interval in a single step, rather than approximating it through multiple steps with a numerical method.

In the computing structure herein adopted, two different events may trigger an update to a neuron's state: a presynaptic spike event or a monitor event. For a presynaptic spike event, the receiving neuron states must first be updated to the current clock time considering a null input, and then updated according to the input current induced by the spike event. After these updates, conditions for postsynaptic spike generation must be checked, and, if satisfied, a new spike event is scheduled for the corresponding postsynaptic neurons. Conversely, monitor events do not interfere with the network's dynamics; instead, they trigger state updates across all neurons, ensuring that the overall network state is valid for measurement by updating it to the current clock time.

3.4 Work Plan

Figure 6 presents the planned implementation schedule for this project. This plan is divided into 3 distinct steps: foundation, refinement and application. Following this implementation plan, a time buffer is reserved to finalize the project deliverables (e.g. documentation). Nevertheless, the deliverables were developed iteratively throughout the entire project, enabling their incremental construction.

During the foundation phase, the main features provided by the simulator were implemented, allowing for the simulation of neurons and basic neural networks. In this phase, research had to be performed on existing implementations, providing a foundation for the design choices herein adopted. With this information, a prototype of the simulation kernel and its corresponding *Python* interface were developed. However, it is important to remark that continuous research was carried out throughout the entire project schedule in order to stay up to date with the state of the art.

In the refinement phase, the initial implementation was extended to satisfy all functional and non-functional requirements. This involved establishing a baseline for comparison with existing simulators and optimizing performance.

Lastly, during the application phase, the simulator was used to address machine learning tasks. This was done within the reservoir computing paradigm by constructing a liquid state machine from a simulated network of neurons. Specifically, the N-MNIST (Orchard et al., 2015) and Spiking Heidelberg Digits (SHD) (Cramer et al., 2022) classification tasks were treated using this liquid state machine.

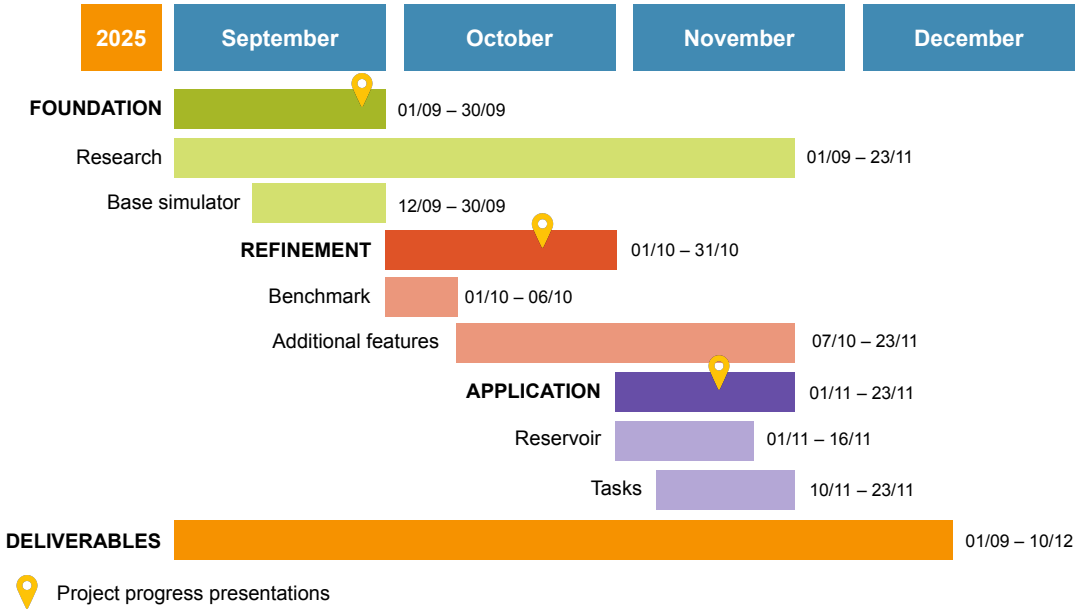


Figure 6 – Project schedule (Gantt chart).

4 Implementation

This chapter describes the overall structure of the *C++* kernel and *Python* interface, discussing design choices. Use cases, advantages and drawbacks of this implementation are also discussed here.

4.1 Simulation Kernel

In this project, an event-based simulation paradigm is adopted. This choice is based on two main factors. First, event-based simulations have been shown to be computationally efficient for sparse spiking activity, as they avoid unnecessary state updates to neurons not affected by some event at the current time (Mattia; Giudice, 2000). Second, event-based simulations allow for high temporal precision, as errors introduced by spike time alignment in step-based simulators are avoided.

Nevertheless, the event-based approach presents a few limitations, such as the restriction of neuron models to those compatible with event-based updates, which excludes some complex models, such as the Hodgkin-Huxley model (Hodgkin; Huxley, 1952). Furthermore, it tends to restrict parallelism, as events must be processed in order. Despite these limitations, the advantages outweigh the drawbacks for the intended applications of this simulator, due to the potential performance gains under sparse spiking activity (Mattia; Giudice, 2000).

The simulation kernel was designed around an event queue, implemented with a binary heap data structure. Events are ordered by their scheduled time, allowing for efficient retrieval and insertion while maintaining the execution order, that is, with a time complexity of $O(\log n)$ for both operations. The `priority_queue` container from the *C++* standard library was used to implement the heap structure.

Two distinct event types are handled by the simulator: spike events and update events. Spike events represent the occurrence of an incoming spike to a neuron, which may cause a change in its state variables. These events can either be created externally or generated internally by synaptic connections. Update events, on the other hand, represent a simple update of the network state variables at a specific time. Since updates only occur when an event is processed, update events are necessary to ensure that neurons are updated correctly before their state variables are read.

An object-oriented approach was used to develop this implementation of the spiking neural network simulator kernel. Figure 7 provides an UML class diagram, detailing the main classes and how they are associated.

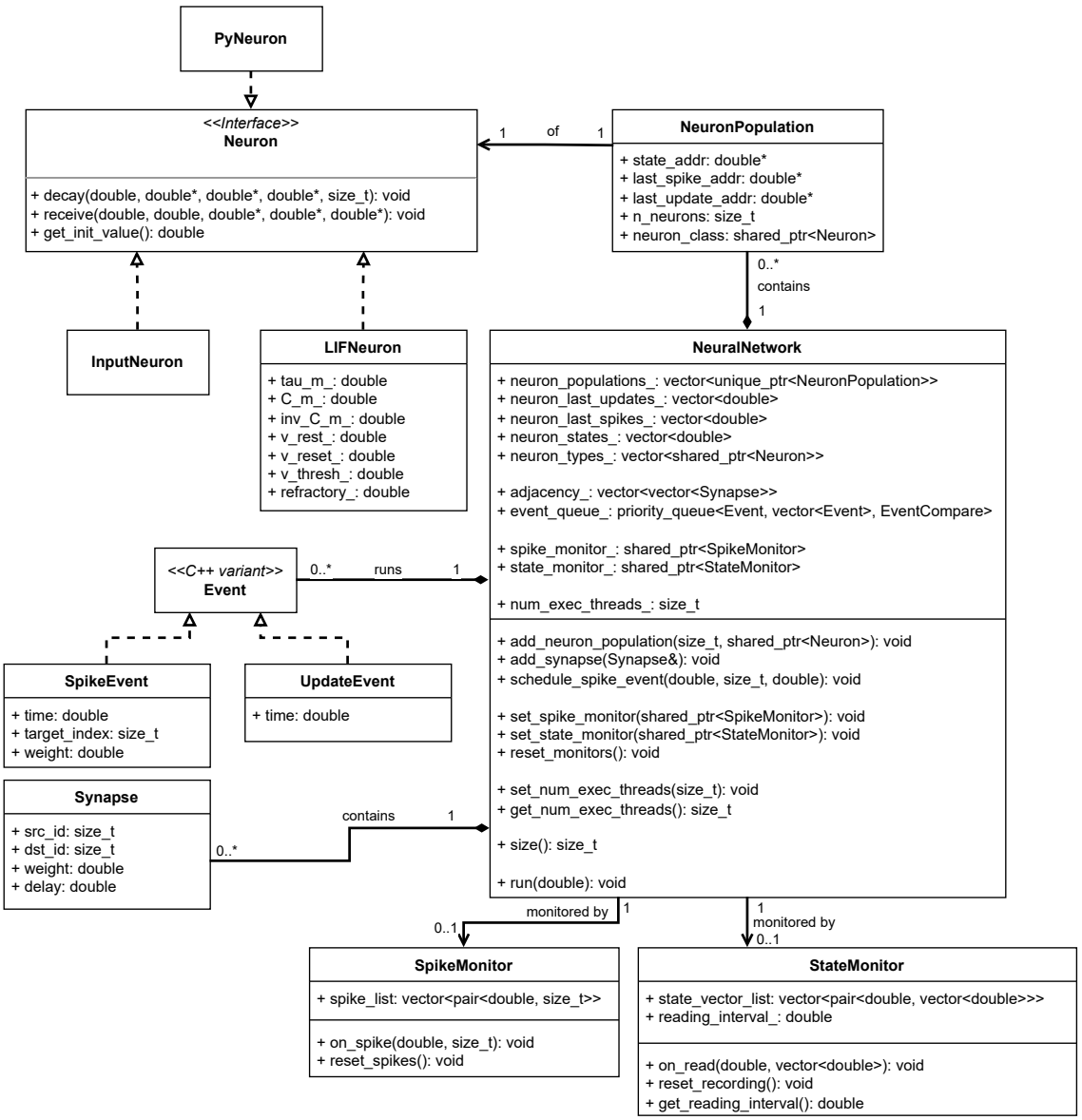


Figure 7 – UML class diagram of the spiking neural network simulator kernel.

A few design choices are worth mentioning. Firstly, the `Neuron` class is an abstract interface class, which defines the common interface for all neuron models. Implemented neuron models must be compatible with event-based updates, as the interface class is unfit for step-based updates. Currently, the only implemented neuron model is the LIF neuron (`LIFNeuron` class).

Secondly, the `NeuralNetwork` class is responsible for the simulation environment, managing the neurons and synapses, as well as the event queue. This class provides methods to add neurons and synapses, schedule external inputs, and run simulations. Furthermore, this class contains all arrays holding the neuron state variables, as well as update and spike times. This centralization allows for contiguous memory allocation in a structure of arrays (SoA) configuration, which allows for better cache performance and the

use of SIMD (Single Instruction, Multiple Data) instructions. Due to this design choice, the neuron classes do not contain state variables, only methods to update them. Neurons with similar types and parameters are grouped into `NeuronPopulation` objects, which facilitate parallelism and the management of large networks.

Lastly, the simulation outputs are not retrieved directly from the `NeuralNetwork` class. Instead, a `SpikeMonitor` class is used to manage the recording of spikes during the simulation, while a `StateMonitor` class is used to record the state variables of neurons at specified intervals. In both cases, the experimental data is retrieved into the monitors as the simulation is executed. This separation of concerns allows for cleaner code and easier management of experimental outputs.

It is important to remark that the state monitor triggers update events to be inserted into the simulator's event queue, as the state variables must be updated before being recorded. Hence, an excessive read rate may degrade simulation performance, overshadowing the gains of an event-based approach.

The event-based simulation procedure is summarized in Figure 8. The algorithm start indicated here corresponds to the point at which the `NeuralNetwork.run(double T)` method is invoked, in which `T` denotes the maximum simulation time. Note that the user must populate the event queue with spike events beforehand; the state monitor generates update events during the setup phase of the `run` method.

To interface external inputs with the network, an auxiliary class, `InputNeuron`, is introduced: an artificial neuron type which transmits a spike to all post-synaptic neurons whenever a pre-synaptic spike is received. This class primarily provides a mechanism to transmit a single input spike to multiple neurons within the simulation kernel, which is useful for defining an input layer between the network and external inputs. Such an approach is notably more efficient than defining an input layer in Python and scheduling the resulting spike events manually.

In order to evaluate the correctness of the implementation, a suite of unit tests was developed using the *GoogleTest* framework. These tests cover the main functionalities of the kernel through small simulation scenarios.

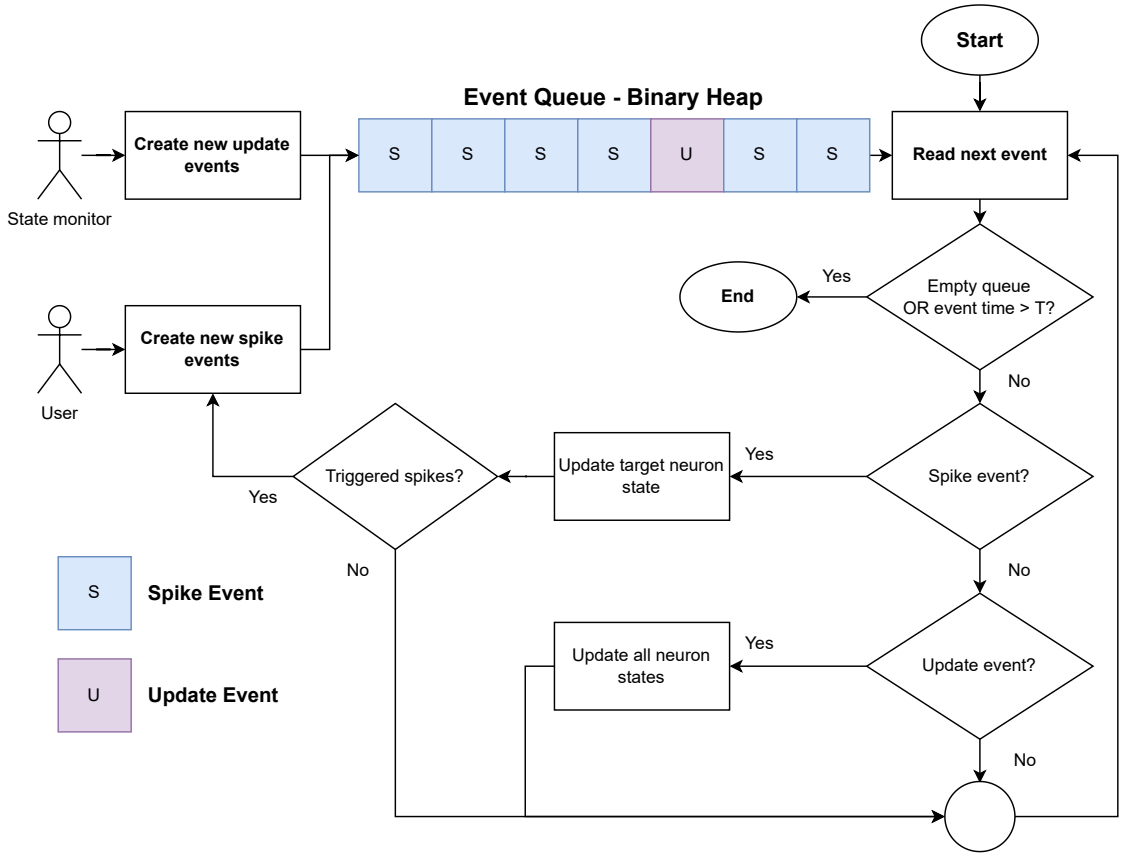


Figure 8 – Diagram of the event-based simulation algorithm, beginning with the simulation run call.

4.2 Python Interface

The *Python* interface was developed using the *pybind11* library, which allows for seamless interoperability between *C++* and *Python*. This interface was implemented by exposing the main kernel classes to *Python*. Some adaptations were made to allow for compatibility, such as the creation of the `PyNeuron` helper class, needed for the abstract neuron interface to work in *Python*. Furthermore, as operations happen over contiguous arrays, the *C++* standard library vectors were mapped to *NumPy* arrays, allowing for user-friendly data exchange between the two languages.

The interface allows for the construction of spiking neural networks in a gradual manner, adding neurons and synapses step by step. Monitors can also be instantiated and added to the network, setting the desired recording parameters. Once the network is fully constructed, the simulation can be executed for a specified time duration. After the simulation, the recorded data can be retrieved from the monitors as *NumPy* arrays.

An example of the usage of the *Python* interface is provided in Figure 9, depicting the construction and simulation of two neurons connected by a synapse.

It is important to emphasize that the methods and classes exposed through the

```python
import pysnnblaze as pb

num_threads = 1      # number of threads to use for tests

tau_m = 20e-3        # membrane time constant
C_m = 1              # membrane capacitance - unitary for simplicity
# Remark: implies that potential changes are equal to the spike weights
v_rest = -70e-3      # resting potential
v_reset = -70e-3     # reset potential after spike
v_thresh = -50e-3    # threshold
refractory = 5e-3    # refractory period
neuron = pb.LIFNeuron(tau_m, C_m, v_rest, v_reset, v_thresh, refractory)

monitor = pb.SpikeMonitor()

# Setting up the neural simulator
nn = pb.NeuralNetwork()
nn.set_num_exec_threads(num_threads)

# Building a neural network
nn.add_neuron_population(2, neuron)
# Creating a synapse with a 20 ms propagation delay and 40 mV weight
syn = pb.Synapse(0, 1, 40e-3, 20e-3)
nn.add_synapse(syn)
# Setting a monitor to read output spikes
nn.set_spike_monitor(monitor)
# Scheduling an input spike event to neuron 0 at 50 ms (40 mV weight)
nn.schedule_spike_event(50e-3, 0, 40e-3)

# Running the simulation for 100 ms
nn.run(100e-3)
# Showing observed spike events
print(monitor.spike_list)
```

Figure 9 – Demonstration of the *Python* interface for constructing and simulating two neurons connected via a single synapse.

Python interface retain the same names and signatures as in the *C++* kernel. Thus, the *Python* interface acts as a direct bridge between the user and the compiled machine code obtained from the *C++* implementation.

Having constructed the basic features necessary for simulating a spiking neural network, as well as an effective *Python* interface, it can be applied to various use cases, such as computational neuroscience research, machine learning applications, and educational purposes. The interface remains flexible, allowing for modifications and functionality extensions.

4.3 Classes and Methods

This section describes the classes and methods provided by the simulator. Naming conventions and function signatures are consistent across the *Python* and *C++* interfaces. Since specific data types are defined in the *C++* interface, this version is presented in detail here.

4.3.1 Neuron Classes

The `Neuron` class defines the abstract interface that all neuron models must implement. It is centered around a structure-of-arrays (SoA) representation to support SIMD-friendly operations and efficient cache usage. State updates must follow a lazy-evaluation strategy, that is, the neuron is only updated when perturbed by an incoming spike or when an update is explicitly requested.

This interface exposes the following methods:

- `decay(double t, double* state, double* last_spike, double* last_update, size_t n)`:
  Updates the neuron state vector (`state`) to time `t` using a decay rule, which takes into consideration the times indicated by `last_spike` and `last_update`. Operates over `n` contiguous elements.
- `receive(double t, double charge, double* state, double* last_spike, double* last_update)`:
  Processes a synaptic input of magnitude `charge` at time `t`. Updates the state accordingly, taking into consideration `state`, `last_spike` and `last_update`, and returns `true` if the neuron emits a spike. Operates over a single element.
- `get_init_value()`:
  Returns the initial value used to initialize the neuron state in the simulator.

The `InputNeuron` class does not operate over its state variables, only serving as a hub to distribute spike events (triggered regardless of input magnitude). In contrast, the `LIFNeuron` implements the LIF neuron model with the characteristics described in Section 2.1.

The `NeuronPopulation` struct represents a homogeneous group of neurons sharing the same model. It stores contiguous memory addresses for the states (`state_addr`), last spike times (`last_spike_addr`), and last update times (`last_update_addr`), enabling SIMD operations consistent with the simulator's structure-of-arrays layout. Each population holds a `Neuron` implementation (`neuron_class`) that defines the associated neural model

and its parameters. The `n_neurons` field specifies the size of the population, determining the length of the associated arrays.

4.3.2 Monitors

The `SpikeMonitor` class can be associated to a `NeuralNetwork` object to record spike events emitted during a simulation. It stores time-neuron pairs in a contiguous list for efficient retrieval from both C++ and Python.

This class provides the following methods:

- `on_spike(double time, size_t neuron_id)`:
  Appends a spike event, consisting of a `(time, neuron_id)` pair, to the internal list.
- `reset_spikes()`:
  Clears the stored spike list, removing all recorded events.

The `StateMonitor` class can be linked to a `NeuralNetwork` object to record instantaneous neuron population states at fixed time intervals. Its public attribute `state_vector_list` contains the recorded snapshots, with each entry storing the sampling time together with the full network state vector. The attribute `reading_interval` specifies the sampling period.

The available methods are as follows:

- `on_read(double time, std::vector<double> state_vector)`:
  Stores a state recording by appending the pair `(time, state_vector)` to the internal list.
- `reset_recording()`:
  Clears all stored state vectors.
- `get_reading_interval()`:
  Returns the fixed sampling interval used by the monitor.

4.3.3 Synapses and Events

The `Synapse` class stores the properties of a neural connection. The attributes `src_id` and `dst_id` identify the presynaptic and postsynaptic neurons, respectively. The attribute `weight` specifies the synaptic weight of associated spikes, while `delay` represents the transmission latency before the spike reaches the target neuron.

The `SpikeEvent` class represents the delivery of a spike to a target neuron. Its attributes store the spike arrival time (`time`), the index of the postsynaptic neuron receiving the spike (`target_index`), and the synaptic weight applied during the update (`weight`).

The `UpdateEvent` class represents a scheduled state update. It contains a single attribute, `time`, indicating when the network state update is scheduled to occur.

4.3.4 Neural Network

The `NeuralNetwork` class is responsible for managing the event-driven neural simulation. The public attribute `sim_time` stores the current simulation time, allowing sequential runs. Internally, neuron populations are stored in `neuron_populations_`. The vectors `neuron_states_`, `neuron_last_spikes_`, and `neuron_last_updates_` contain contiguous state information for all neurons in the network. The array `neuron_types_` provides fast lookup of the neuron model associated with each neuron index. Synaptic connectivity is defined in `adjacency_`, a per-neuron list of outgoing synapses. The `event_queue_` is a priority queue that stores pending events sorted by their scheduled execution time. The attributes `spike_monitor_` and `state_monitor_` are optional monitors used to record spike trains and state vectors during simulation. Finally, `num_exec_threads_` specifies the number of threads used for parallel sections of the implementation.

The methods are defined as follows:

- `add_neuron_population(size_t size, std::shared_ptr<Neuron> neuron_type)`:
  Creates a new neuron population of the given `size` using the specified neuron model (`neuron_type`).
- `add_synapse(const Synapse& synapse)`:
  Adds a synaptic connection (`synapse`) to the adjacency structure.
- `schedule_spike_event(double time, size_t neuronIndex, double weight)`:
  Schedules an external spike event at `time`, which targets `neuronIndex` with a value of `weight`.
- `set_spike_monitor(std::shared_ptr<SpikeMonitor> monitor)`:
  Attaches a spike `monitor` to record emitted spikes.
- `set_state_monitor(std::shared_ptr<StateMonitor> monitor)`:
  Attaches a state `monitor` to record state vectors.
- `set_num_exec_threads(size_t n)`:
  Sets the number of threads used for parallel operations to `n`.
- `get_num_exec_threads() const`:
  Returns the currently configured number of execution threads.
- `run(double T)`:
  Executes the event-driven simulation for `T` seconds.

- `reset_monitors()`:
  Clears all data from attached monitors.

- `size() const`:
  Returns the total number of neurons across all populations.

4.4 Simulator Usage

The simulator is primarily designed to be used through the high-level *Python* interface, although direct interaction with the *C++* simulation kernel is also possible. A typical simulation procedure consists of the following steps:

1. instantiating neuron models and specifying their parameters;
2. creating a network object that encapsulates the simulation logic;
3. configuring the network object and setting up monitors;
4. adding populations of pre-instantiated neuron models;
5. building synaptic connections between the neurons;
6. scheduling input events according to the experiment setup;
7. running the experiment for a specified duration;
8. retrieving the generated data.

This ordering is not strict, as configurations can be changed and monitors can be added until the experiment is effectively run.

Given the developed interface, a stereotypical simulator use case is depicted in Figure 10. In this scenario, a network composed of two distinct LIF neuron populations and an input neuron is built and simulated. Both their internal state and generated spike events are monitored.

This experiment makes use of all features offered by the simulator, hence it is a good illustration of the expected user interaction with its interface. The interaction between this actor and the simulator is formalized in the Unified Modeling Language (UML) sequence diagram depicted in Figure 11. In most cases, a similar interaction will take place, although some features may not be used; for instance, input neurons might not be included, or no state monitor may be assigned. Resetting the monitors is unnecessary for the first simulation run, but becomes useful when performing multiple runs on the same network instance.

```python
# Initializing a network and monitors
example_net = pb.NeuralNetwork()
spike_monitor = pb.SpikeMonitor()
state_monitor = pb.StateMonitor(reading_interval=0.1e-3)
spike_monitor.reset_spikes()

lif_neuron1 = pb.LIFNeuron(tau_m=20e-3, C_m=1, v_rest=-70e-3, v_reset=-70e-3,
    v_thresh=-50e-3, refractory=2e-3)
lif_neuron2 = pb.LIFNeuron(tau_m=60e-3, C_m=2, v_rest=-70e-3, v_reset=-70e-3,
    v_thresh=-50e-3, refractory=2e-3)
inp_neuron = pb.InputNeuron()

# Adding neuron populations
example_net.add_neuron_population(1, inp_neuron)  # index 0
example_net.add_neuron_population(2, lif_neuron1) # index 1,2
example_net.add_neuron_population(1, lif_neuron2) # index 3

# Adding synapses
synapses = [
    pb.Synapse(0, 1, 10e-3, 5e-3),
    pb.Synapse(1, 2, 15e-3, 5e-3),
    pb.Synapse(2, 3, 20e-3, 5e-3),
    pb.Synapse(3, 1, 10e-3, 5e-3),
]
for syn in synapses:
    example_net.add_synapse(syn)

# Setting monitors
example_net.set_spike_monitor(spike_monitor)
example_net.set_state_monitor(state_monitor)

# Scheduling 50 spike events for input neuron
for i in range(50):
    example_net.schedule_spike_event(0.002*i, 0, 20e-3)

# Running the network for 200 ms
example_net.run(0.2)
```

Figure 10 – Demonstration of the *Python* interface for the simulation of a network composed of two distinct neuron populations and an input neuron.

The network built for this experiment is illustrated in Figure 12. Note that it is composed of three distinct population: a single input neuron (index 0), two LIF neurons with one parameter set (indexes 1 and 2), and a last LIF neuron with another parameter set (index 3). All inputs are mapped exclusively to the input neuron, which connects to neuron 1. Neuron 1 then connects to neuron 2, neuron 2 to neuron 3, and neuron 3 back to neuron 1, forming a recurrent loop.

After executing the experiment, the results can be easily retrieved as *NumPy* arrays by accessing monitor attributes `spike_list` from `SpikeMonitor` and `spike_vector_list`

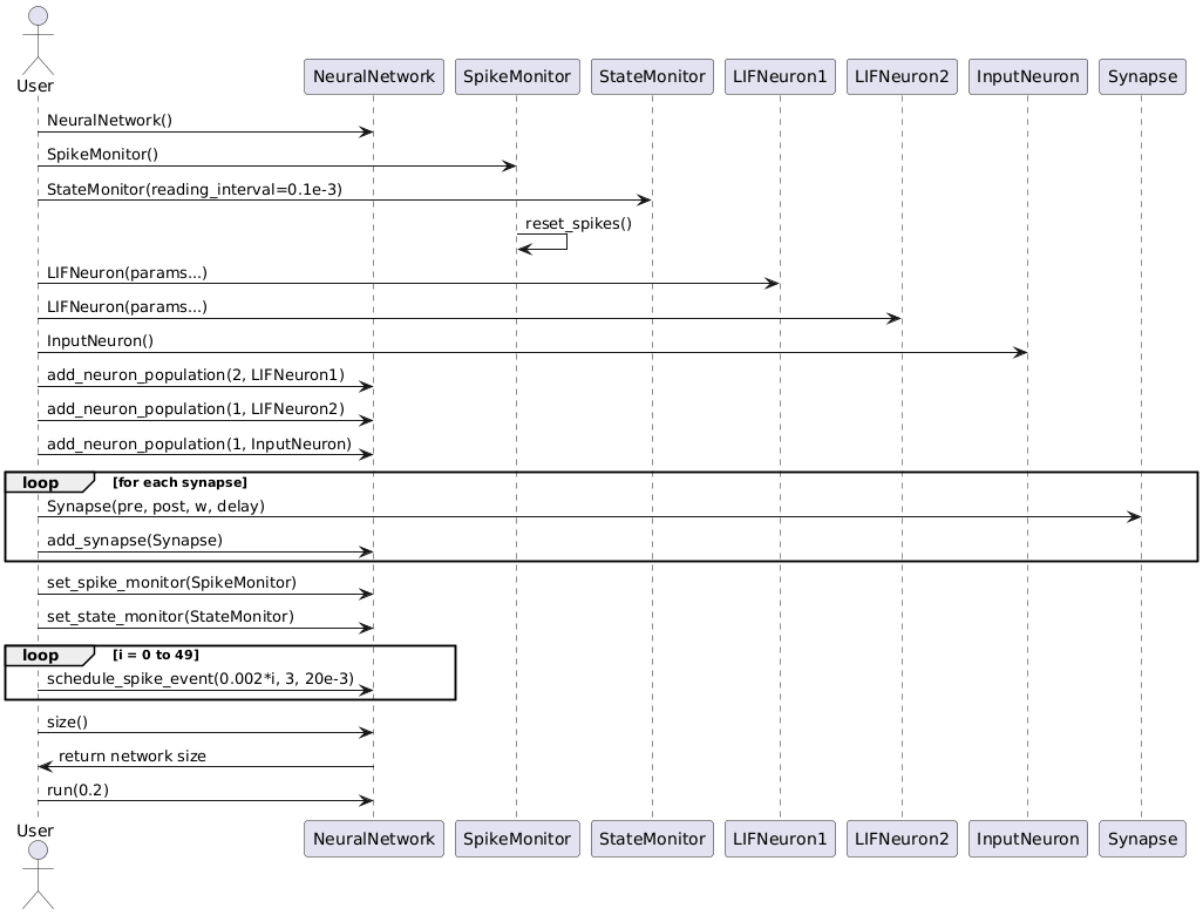


Figure 11 – Sequence diagram for a typical simulation workflow with the developed simulator.

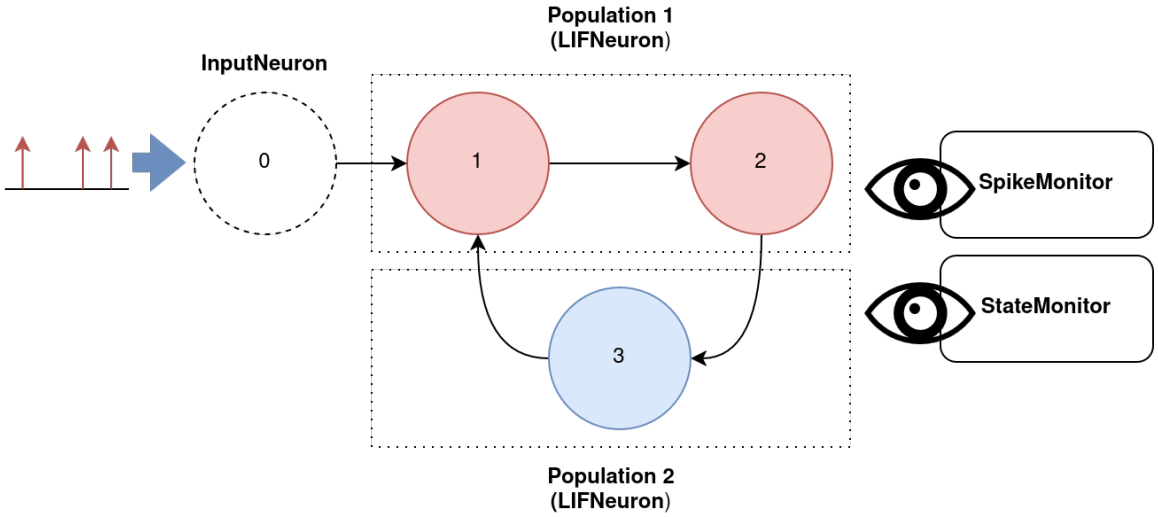
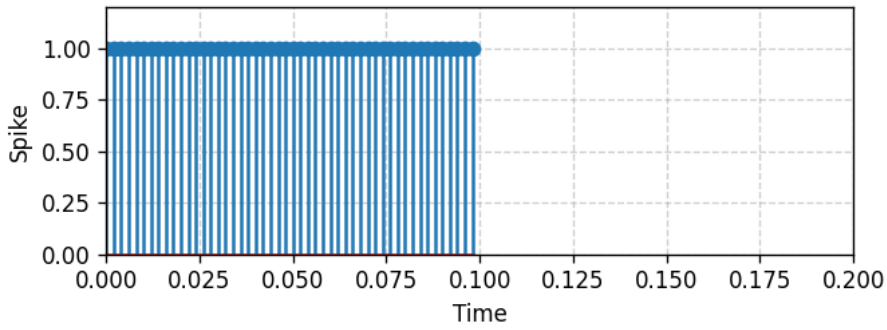
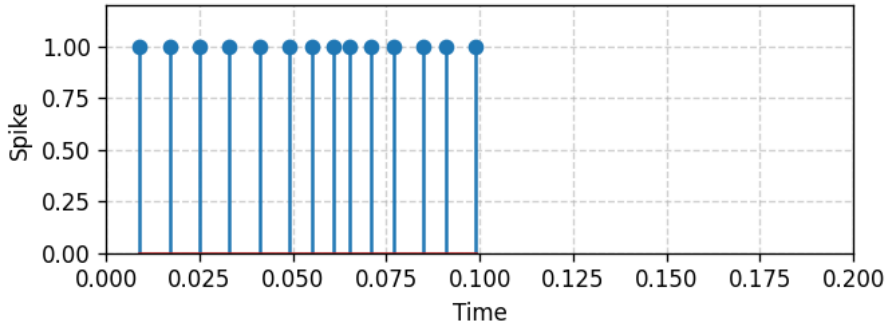


Figure 12 – Architecture of a simulated network, featuring one input neuron and two LIF neuron populations.

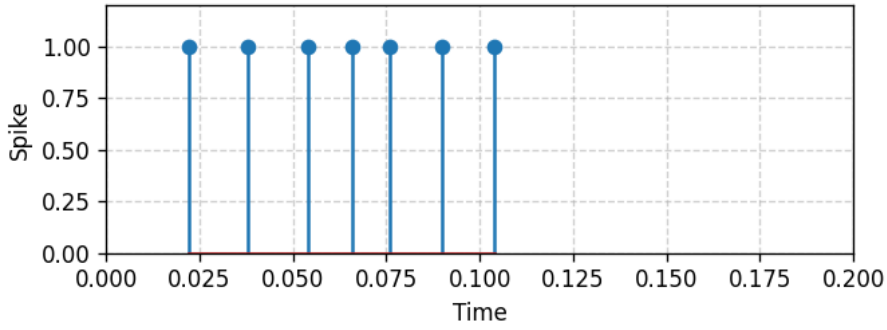
from `StateMonitor`. For this experiment, the observed spikes and states were plotted over the simulated time period, yielding Figure 13 and Figure 14, respectively. In these preliminary results, the expected behavior of LIF neurons can be observed, such as exponential decay over time, increases in membrane potential upon pre-synaptic spikes, and resets to the resting potential when the threshold is exceeded and post-synaptic spikes are generated.



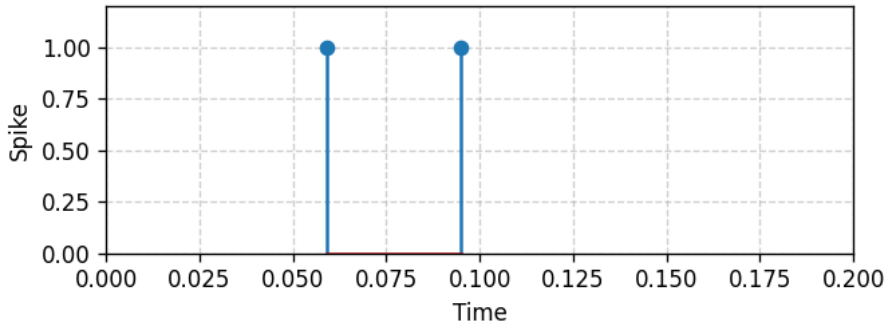
(a) Post-synaptic spikes of the input neuron (index 0), reproducing the input spike train.



(b) Post-synaptic spikes of neuron 1.

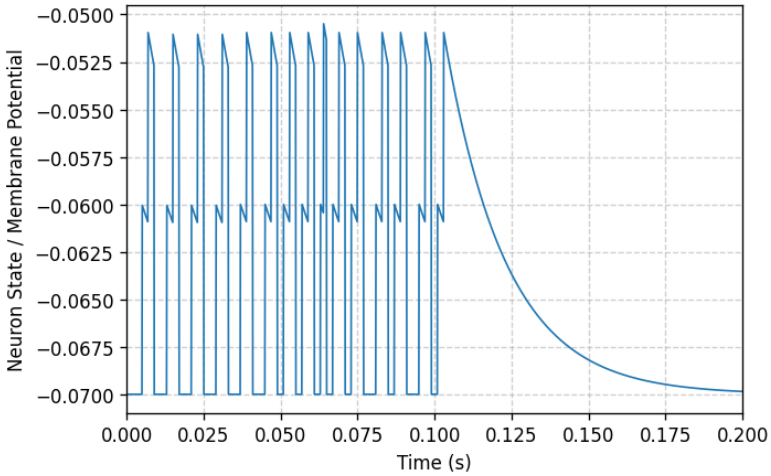


(c) Post-synaptic spikes of neuron 2.

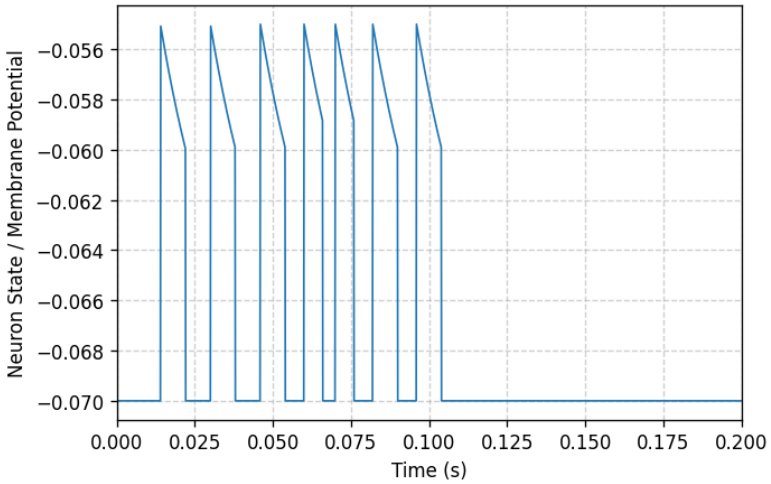


(d) Post-synaptic spikes of neuron 3.

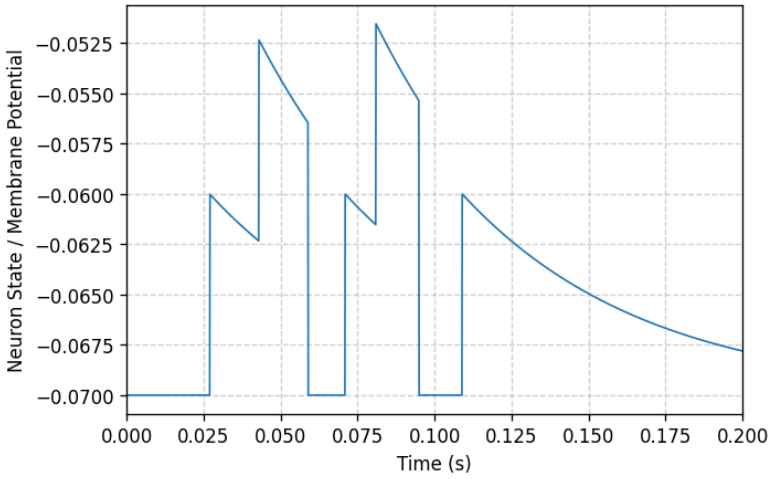
Figure 13 – Post-synaptic spike activity recorded for each neuron within the example neural network.



(a) Membrane potential of neuron 1.



(b) Membrane potential of neuron 2.



(c) Membrane potential of neuron 3.

Figure 14 – States (membrane potentials) recorded for each neuron within the example neural network.

4.5 Parallelization Strategy

Due to the inherently sequential execution of the event-based simulation procedure used in this project, parallelism is not easily exploitable in the current implementation. Mainly, neuron state updates could be performed in parallel when possible, but in the current approach such simultaneous updates occur only when updating the network for a state monitor reading.

In such scenarios, *OpenMP* directives are used to implement parallelism. Neuron updates are always performed using available SIMD instructions, which depend on the compiler; this is enabled by allocating state variables in contiguous vectors. Additionally, multithreading can optionally be enabled by calling the `NeuralNetwork` method `set_num_exec_threads(size_t n)` with $\texttt{n} > 1$. It must be noted that, in this case, the network size must be sufficiently large for parallelism to yield a performance gain that exceeds the overhead introduced by multithreading.

In conclusion, parallelism plays a limited role in improving the simulator's performance in the current implementation, given that it only affects state updates used for monitoring. Consequently, all experiments conducted in this work are executed on a single thread. To achieve further parallelism gains, the execution paradigm would need to be changed.

An option to increase the viable parallel sections would be to discretize time, allowing the grouping events that may be executed simultaneously. Distributing these update tasks over a thread pool would then enable parallelism in the event-based simulation procedure. Here, this design choice is not applied, as it would lead to a loss of precise event timing. Nevertheless, it remains an interesting opportunity to be explored in future works.

5 Experiments and Results

This chapter describes experiments performed with the simulator presented in previous sections, as well as the results obtained. Initially, a comparison with other existing implementations was developed, allowing for a comparison in terms of global behavior and computational efficiency. Additionally, a study of the simulator's applicability to constructing liquid state machines capable of solving machine learning tasks was conducted, demonstrating some of its potential use cases.

5.1 Correctness Assessment

To validate the correctness of the simulator implementation, a few simple network simulations were performed. These initial experiments seek to assert the correctness of this implementation through a comparison with well consolidated simulators; here, a comparison is performed with *Brian 2* (Stimberg; Brette; Goodman, 2019) and *NEST* (Diesmann; Gewaltig, 2001), two widely used spiking neural network simulators.

A random graph is adopted as the network structure for these experiments, with $N = 100$ neurons connected randomly with a connection probability of $p = 0.2$. All neurons are modeled as LIF neurons, with parameters listed in Table 4. 20% of neurons are set as inhibitory (corresponding to the last 20 indices), seeking to balance neural activity. Synaptic weights are set to 1.5 pC for excitatory synapses and -2 pC for inhibitory synapses. Synaptic delays are randomly sampled from a uniform distribution between 1 ms and 2 ms.

Table 4 – Parameters of the leaky integrate-and-fire neuron model for the simple network simulations.

Parameter	**Symbol**	**Value**
Membrane time constant	τ_m	20 ms
Membrane capacitance	C_m	200 pF
Resting potential	v_{rest}	-70 mV
Reset potential after spike	v_{reset}	-70 mV
Spike threshold	v_{thresh}	-50 mV
Refractory period	$t_{\text{refractory}}$	5 ms

This network is randomly generated, and then instantiated in all three simulators. The first 10 neurons (excitatory) are then stimulated with spikes generated according to Poisson processes with a rate of 40Hz.

By adopting this setup, the global activity of the network can be compared between the three simulators, allowing for an assessment of the correctness of the implementation.

The experimental design takes inspiration from the work of Brunel (Brunel, 2000), which studies the dynamics of sparsely connected networks of excitatory and inhibitory neurons, giving a theoretical foundation for the expected behavior of this system. However, certain aspects are modified, such as the model parameters and the stimulation protocol, bringing the network closer to commonly used configurations in reservoir computing. One such choice is the connection of inputs to only a subset of neurons, which is a common practice in LSMs (Maass; Natschläger; Markram, 2002). Though simple, this random network structure has been successfully used to construct LSMs (Wijesinghe et al., 2019).

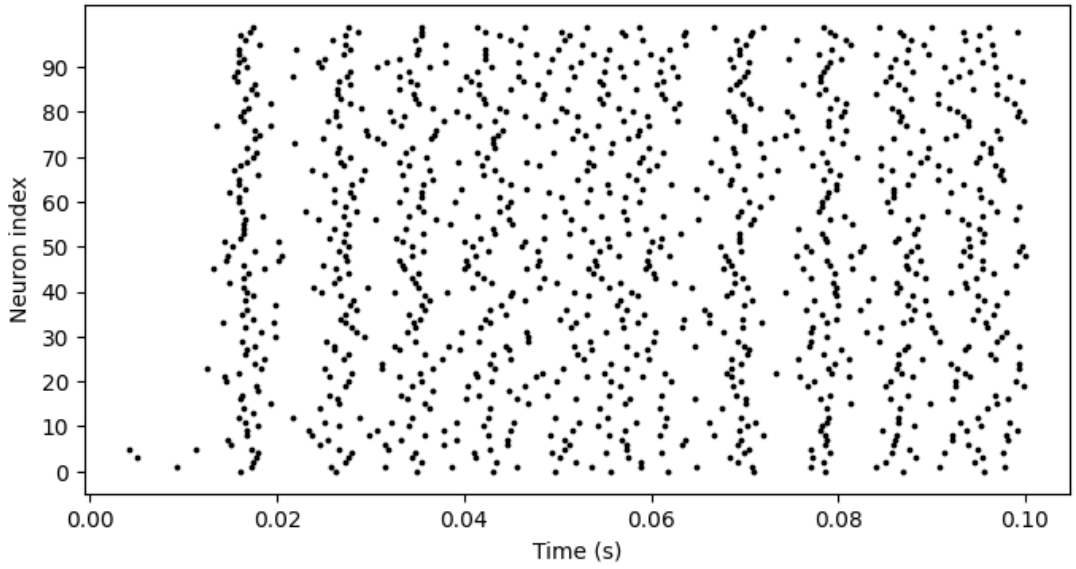
Initially, the network is simulated for a short duration of 100 ms. The resulting raster plots for one instance of this simulation are shown in Figure 15. Raster plots are graphs which indicate the spike times of each neuron in the network as dots or bars, giving a visual overview of the spiking activity.

The results show a good agreement between the three simulators only during the start of the simulation, with an increasing divergence through time. This is evident from the accurate overlap of events (represented as dots) at the initial observed events, followed by a growing discrepancy between this simulator, Brian 2, and NEST. This behavior is expected, as such systems are known to exhibit chaotic behavior (Vreeswijk; Sompolinsky, 1996); that is, small divergences in the numerical implementations get amplified exponentially through time, leading to different spike patterns after a certain period.

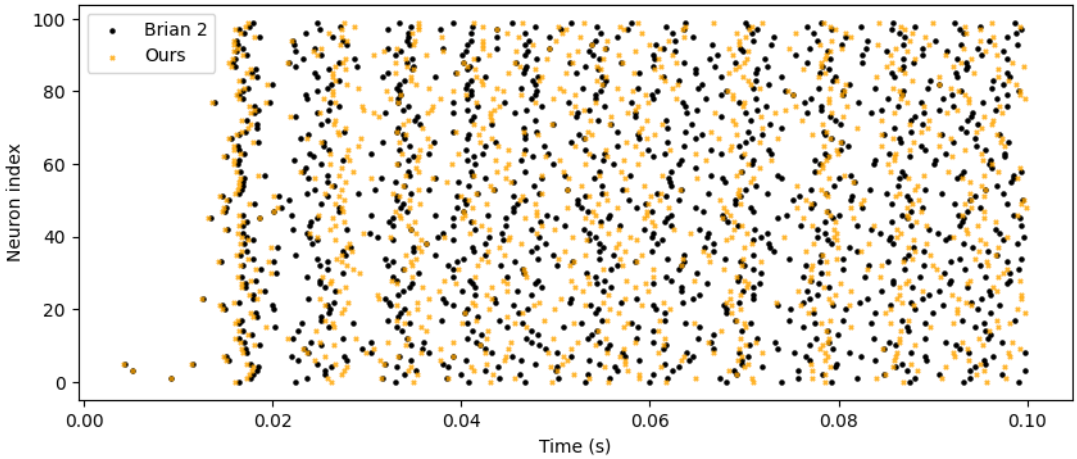
Furthermore, the simulations display a synchronous irregular (SI) firing pattern (Brunel, 2000), characterized by periodic bursts of activity throughout the entire network, which can be observed through the irregular vertical stripes of spikes formed in the raster plots. This behavior is consistent with one of the possible activity patterns for this kind of network, indicating that the implementation captures the essential dynamics of the system in a manner that is consistent with other simulators.

To further assess the agreement between the three simulators, the average firing rate of the network for each neuron is computed over the simulation period. Results are shown in Figure 16, where the firing rates obtained with the proposed simulator are plotted against those obtained with Brian 2 and NEST. The results show a strong correlation between the firing rates obtained with the three simulators, with a mean absolute percentage error of 12.27% between our simulator and Brian 2, and 9.31% between our simulator and NEST. This shows that, despite divergences between simulators, the overall activity levels were consistent across implementations.

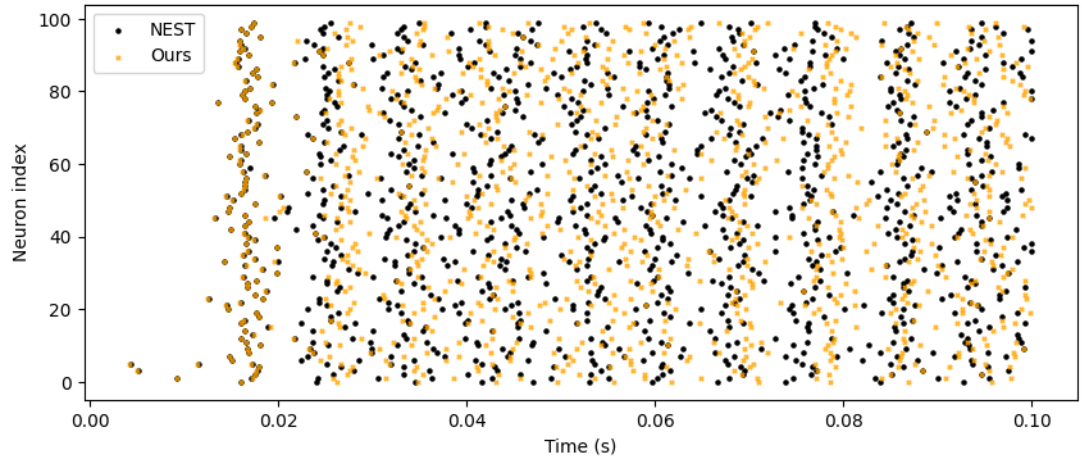
In conclusion, the simulator presents a good agreement with established simulators. While it was impossible to attain identical results when testing sparsely connected networks, this represents an expected impossibility, as the system displays chaotic behavior, presenting different results even among well established simulators. Hence, the results indicate that the implementation is correct, validating it for further experiments.



(a) Our simulation

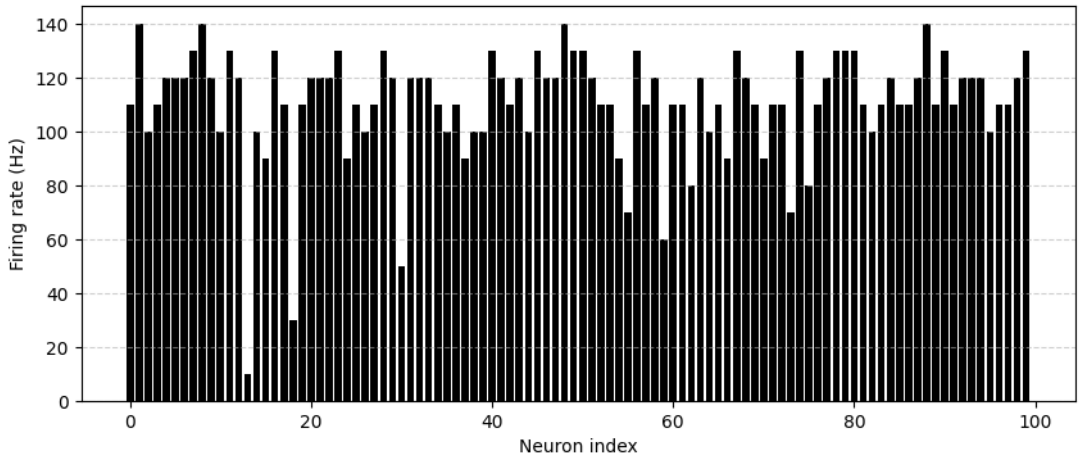


(b) Brian 2 simulation

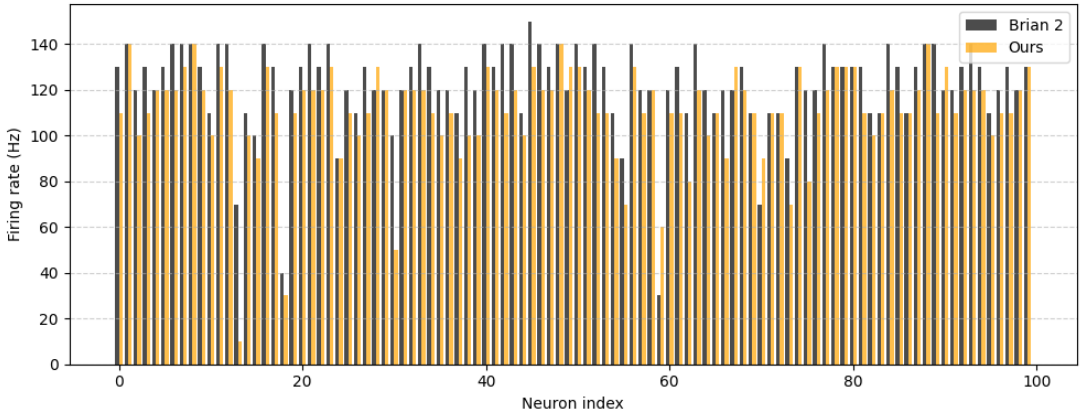


(c) NEST simulation

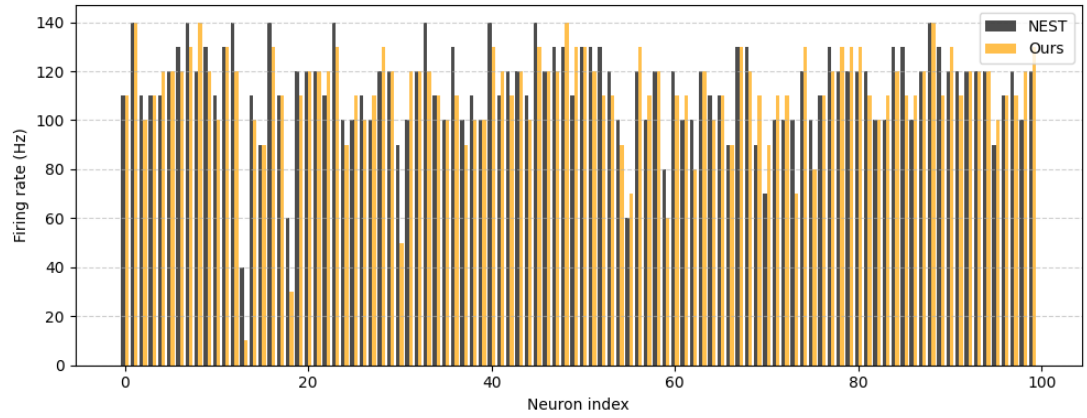
Figure 15 – Comparison of spike rasters for a network of 100 neurons simulated for 100 ms with different simulators.



(a) Our simulation



(b) Brian 2 simulation



(c) NEST simulation

Figure 16 – Comparison of firing rates for a network of 100 neurons simulated for 100 ms with different simulators.

5.2 Performance Benchmarking

To evaluate the computational performance of the proposed simulator, the network used for the correctness test in Section 5.1 was adapted to serve as a benchmark. Firstly, the simulation time was expanded to 10 seconds, providing a more substantial workload for performance measurement. Secondly, the network size was varied, for networks with $N = 100, 200, 400, 800$ neurons, while maintaining the connection probability of $p = 0.2$. This allows for an assessment of how efficiently each simulators scales to larger networks. The stimulation protocol is kept the same as in the correctness test throughout the duration of the experiment. Initially, the experiments are run in a single-threaded configuration.

The experiments are timed according to their setup time, needed to instantiate neurons, create synapses, and run just-in-time compilation when applicable, and the actual execution time. Results are detailed in Figures 17 and 18.

It is noticeable that Brian 2 has a significantly higher setup time compared to both NEST and the developed simulator. This is likely due to Brian 2's just-in-time compilation approach, which introduces overhead during the setup phase when converting networks defined in *Python* into compiled machine code for running experiments (Stimberg; Brette; Goodman, 2019). The proposed implementation also has a significantly lower setup time compared to NEST, likely due to its lightweight and concise implementation, as well as an absence of features that may introduce an additional overhead.

Regarding execution time, our implementation demonstrates performance comparable to NEST for smaller network sizes, but shows worse scaling compared to both NEST and Brian 2, making it less efficient for larger networks. This behavior is likely a consequence of the event-based approach adopted for our implementation, which may introduce a significant sequential workload for managing the event queue as the network increases. Brian 2 showed the best scaling behavior, likely due to its optimized code generation and step-based (time-driven) simulation approach. In contrast, NEST shows a better level of performance compared to Brian 2 for small networks, but it scales poorly compared to it; this may be a consequence of the hybrid approach adopted by NEST, which combines both event-based and time-driven strategies.

In sum, it is possible to assert that the solution proposed here presents competitive performance for small to medium-sized networks, but shows limitations when scaling to larger networks. Further optimizations should be investigated to enhance the scalability of the implementation. Possible directions include adopting alternative event-management strategies or employing hybrid simulation methods similar to NEST.

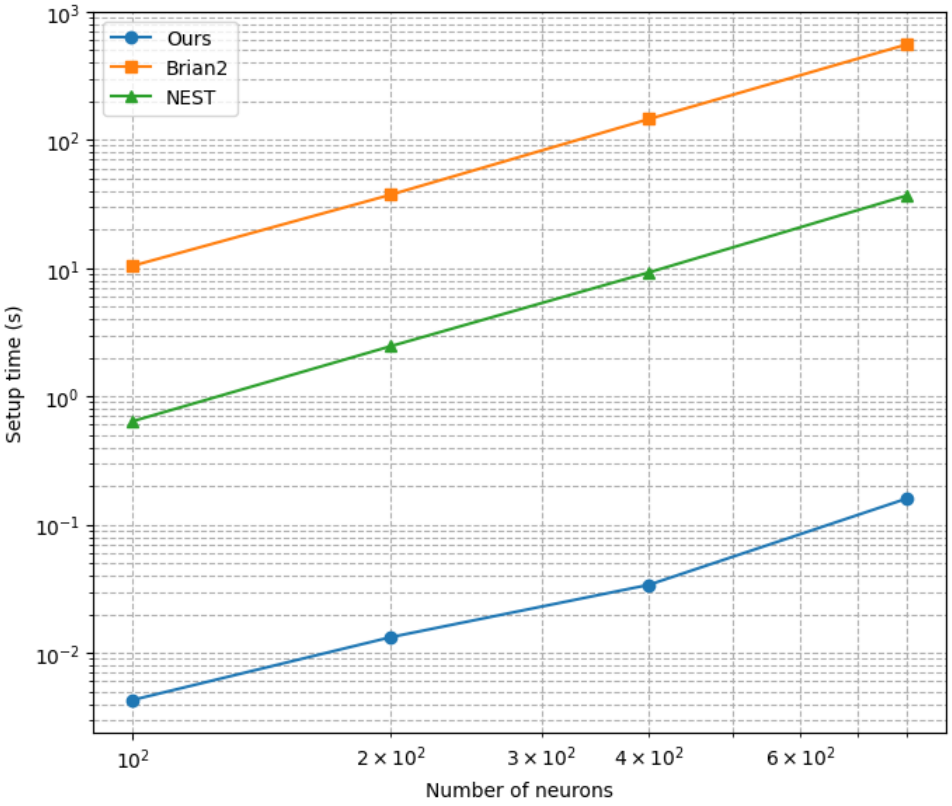


Figure 17 – Setup time for different network sizes across simulators.

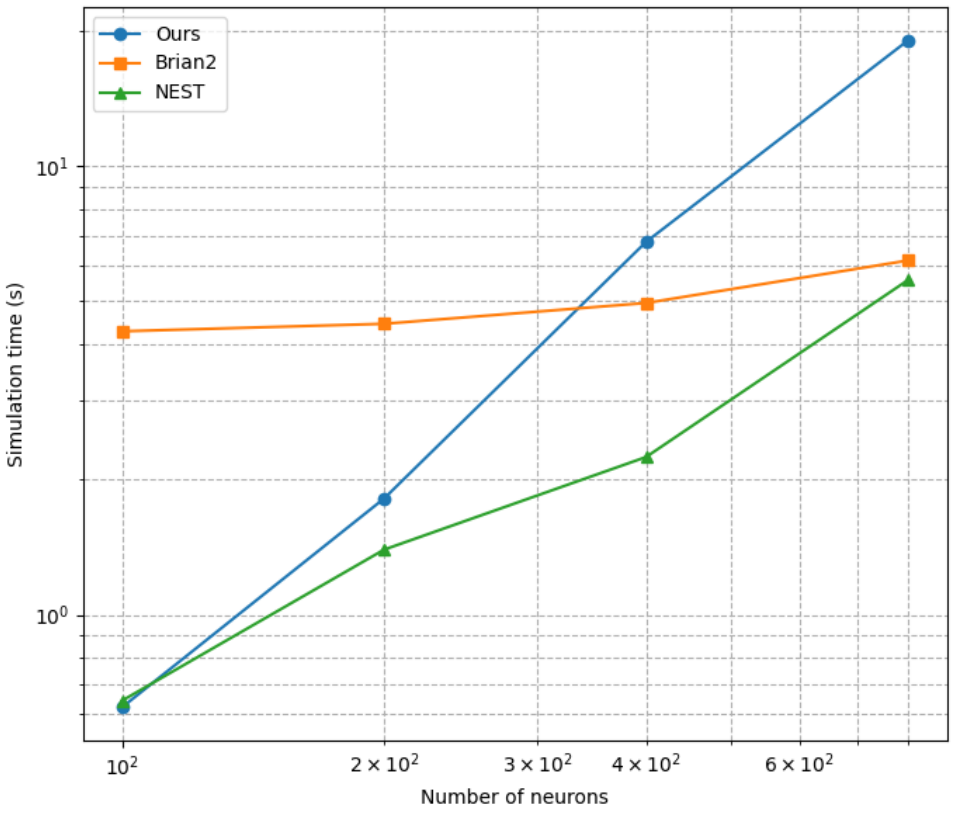


Figure 18 – Simulation time for different network sizes across simulators.

5.3 Constructing Liquid State Machines

In this section, the strategy used to build liquid state machines from the developed neuron and network models is described, along with an assessment of their performance on benchmark datasets.

5.3.1 Network Architecture

The liquid state machine is composed of three main components: the liquid, an input layer, and a readout layer. The liquid, also called the reservoir, is a set of recurrently connected spiking neurons responsible for producing the dynamics that enrich the representation of inputs. The input layer is responsible for mapping incoming spikes into suitable stimuli for the liquid. Lastly, the readout layer is responsible for converting the observed spiking activity into the desired outputs.

5.3.1.1 Recurrent Spiking Reservoir

The adopted liquid architecture takes inspiration from the cortical column model proposed Maass, Natschläger and Markram (2002). The liquid is defined as a three-dimensional grid of LIF neurons, with dimensions (N_x, N_y, N_z). In all experiments, 20% of the neurons are inhibitory and 80% are excitatory, following proportions observed in biological neural systems (Isaacson; Scanziani, 2011; Meinecke; Peters, 1987).

Neurons are randomly connected through a distance-dependent probability function, favoring the formation of local connections. Given two neurons located at positions $a, b \in \mathbb{N}^3$ in the grid, a Gaussian connection probability profile is adopted, defined in Equation (5.1), in which $\|\cdot\|$ represents the Euclidean norm.

$$P_c(x, y) = C_t \cdot \exp\left(-\frac{\|\mathbf{a} - \mathbf{b}\|^2}{\lambda^2}\right) \tag{5.1}$$

The parameters C_t is different for each connection type: excitatory to excitatory (EE), excitatory to inhibitory (EI), inhibitory to excitatory (IE), and inhibitory to inhibitory (II).

Figure 19 displays a visualization of a simple network, with dimensions $N_x = N_y = N_z = 4$, $\lambda = 2$ and $C_{EE} = C_{EI} = C_{IE} = C_{II} = 0.3$, illustrating the spatial arrangement of neurons and their connections.

5.3.1.2 Input Layer

The input layer connects the inputs, which are specific to the dataset being treated, to the liquid. In this case study, a simple connectivity pattern is used, connecting each

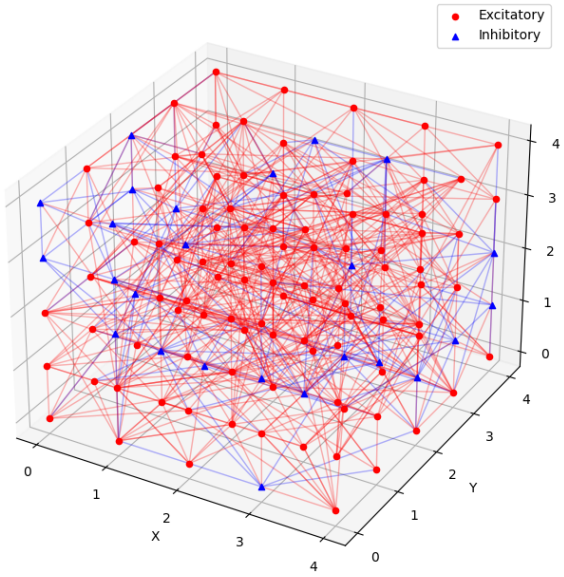


Figure 19 – Example of a reservoir with dimensions $5 \times 5 \times 5$ instantiated according to the described architecture.

available spike input channel to a constant number F_{out} of liquid neurons, sampled with replacement from the set of all neurons.

This approach aims to assign the same influence to all inputs in the network. Variation of the F_{out} parameter allows control over the number of neurons associated with an input channel, thereby controlling the number of neurons whose states can be strongly influenced by the inputs. Purely input-driven dynamics are avoided, since they do not lead to better representations than those provided by the input events.

5.3.1.3 Readout Layer

Two separate readout layers are built, taking different assumptions into consideration. The first scheme is built from the hypothesis that a prediction must be generated for each time instant, given that each dataset sample may be used as an input for multiple time steps. For classification tasks, if a single prediction must be generated over a time interval, a winner-take-all (WTA) strategy is applied to the sequence of predictions, thereby this method is called WTA classifier. The second approach is proposed by building an aggregate representation for each dataset sample through time, by counting the number of spikes generated by each reservoir neuron within the sample's designated time window, producing a single feature vector per sample. Here, the method employing these count-based vectors is referred to as spike-count classifier.

To construct the WTA readout method, spikes generated by a neuron l in the liquid, for the entire duration of the simulation, are interpreted as trains of Dirac deltas $s_l(t)$, with spikes occurring at times $t_i^{(l)}$, as shown in Equation (5.2).

$$s_l(t) = \sum_i \delta\left(t - t_i^{(l)}\right) \tag{5.2}$$

This format is not adequate for classification, hence, different schemes can be adopted to convert these events into a continuous representation. Here, to build a continuous representation through time, these spike trains are convolved with an exponential kernel $k(t)$, as depicted in Equation (5.3).

$$k(t) = H(t)\, e^{-t/\tau} \tag{5.3}$$

In this expression, τ is a time constant and $H(t)$ is the Heaviside step function. Hence, the continuous representation $x_l(t)$ for a neuron n is defined in Equation (5.4).

$$x_l(t) = (s_l * k)(t) = \sum_i k\Big(t - t_i^{(l)}\Big) \tag{5.4}$$

For a network composed of N neurons, the same procedure is applied to all spike trains $\mathbf{s}(t) = (s_0(t) \dots s_N(t))^\top$, in which $\top$ represents the vector transpose operation. Thus, the final representation is illustrated in Equation (5.5).

$$\mathbf{x}(t) = \begin{pmatrix} \sum_i k\Big(t - t_i^{(1)}\Big) \\ \vdots \\ \sum_i k\Big(t - t_i^{(N)}\Big) \end{pmatrix} \tag{5.5}$$

Here, this operation must be adapted to a discrete convolution, which can be handled by a digital computer. By discretizing time using a step Δt, so that $t = n\Delta t$ with $n \in \mathbb{N}_0$. The spike train can then be represented as a discrete sequence, which may be stored in an array, as described in Equation (5.6).

$$s_l[n] = w_l[n]/\Delta t \tag{5.6}$$

With $w_l[n]$ representing the number of spikes generated by neuron l for which $t \in [n\Delta t,\ (n+1)\Delta t)$.

The kernel is also sampled at discrete times, resulting in $k[n]$, as depicted in Equation (5.7).

$$k[n] = k(n\Delta t) = H(n\Delta t)\, e^{-n\Delta t/\tau} \tag{5.7}$$

The discrete convolution, therefore, yields $x_l[n]$, as shown in Equation (5.8).

$$x_l[n] = \sum_{m=0}^{n} s_l[m]\, k[n-m] \tag{5.8}$$

In conclusion, this discrete convolution is applied to all liquid neurons, resulting in the state vectors $\mathbf{x}[n] = (x_0[n], \dots, x_N[n])^\top$. Given a total of T time steps for a

full simulation, the vectors are then concatenated into a design matrix $\mathbf{X}$, detailed in Equation (5.9).

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}[0]^\top \\ \mathbf{x}[1]^\top \\ \vdots \\ \mathbf{x}[T-1]^\top \end{pmatrix} \tag{5.9}$$

A single parameter is introduced by this approach, the time constant τ, which must be tuned according to the problem being solved.

The attained set of states is used to produce predictions $\mathbf{y_p}[n]$ for each time step, as shown in Figure (5.10).

$$\mathbf{y_p} = \begin{pmatrix} \mathbf{y_p}[0]^\top \\ \mathbf{y_p}[1]^\top \\ \vdots \\ \mathbf{y_p}[T-1]^\top \end{pmatrix} \tag{5.10}$$

In the current study case, only classification problems are studied, thus the prediction vectors represent the probability of a certain class being detected at a certain time step.

Given that the samples being classified may occur within a time window $[a, b]$, with $0 \leq a < b < T$, it is necessary to define a method for assigning a single class label to the predictions over this interval. In this work, a winner-take-all (WTA) strategy is used, in which the class with the highest predicted value on the observed window corresponds to the window prediction. For K classes, this procedure is formalized in Equation (5.11).

$$\hat{k}_w = \arg \max_{k \in \{1,\dots,K\}} \max_{t \in [a,b]} \mathbf{y_p}[t]_k \tag{5.11}$$

Alternatively, a second approach based on spike counts is proposed. For this method, it is supposed that the time interval corresponding to data sample to be classified is known beforehand. This could represent, for instance, the time interval of an utterance of a spoken digit. Then, the spike counts $c_l[u]$ for each neuron l at a sample u are taken for each given interval. The feature vectors are $\mathbf{c}[u] = (c_0[u] \dots c_N[u])^\top$, such that, given U samples, the design matrix $\mathbf{X}$ is formed as in Equation (5.12).

$$\mathbf{X} = \begin{pmatrix} \mathbf{c}[0]^\top \\ \mathbf{c}[1]^\top \\ \vdots \\ \mathbf{c}[U-1]^\top \end{pmatrix} \tag{5.12}$$

It is important to remark that, in both cases, if single classifications must be produced for specific time intervals from a non-segmented input stream, an additional

algorithm is required to segment the stream into individual samples. For an audio stream, for instance, an energy-based method could be used to separate distinct words. Once the stream is segmented, predictions can be generated using either approach.

In this study, a supervised learning approach is used to map the features vectors into the desired predictions. Thus, for a design matrix $X \in \mathbb{R}^{m \times n}$, there must be a matching target matrix $y \in \mathbb{R}^{m \times o}$, where o is the number of targets variables. Since the study cases are multi-class classifications tasks, the multinomial logistic regression model is chosen to learn mappings from state vectors to classes.

Multinomial logistic regression, also called softmax regression, is a generalization of logistic regression for multi-class classification problems. For this model, it is assumed that the target variable can take two or more discrete values. The conditional probability of each class $k \in 1, \ldots, K$ is modeled by the softmax function, applied to a set of linear predictors, as shown in Equation (5.13).

$$P(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x} + b_k)}{\sum_{j=1}^{K} \exp(\mathbf{w}_j^\top \mathbf{x} + b_j)} \tag{5.13}$$

Thus, in order to tune the parameters $\mathbf{W} = (\mathbf{w_0}, \ldots, \mathbf{w_K})$ and $\mathbf{b} = (b_0, \ldots, b_K)^\top$, a cost function is minimized. For softmax regression, this function is the cross-entropy loss, with an added regularization parameter λ, as detailed in Equation 5.14.

$$J(\mathbf{w}, \mathbf{b}) = -\frac{1}{N} \sum_{i=1}^{N} \log \Big(P(y_i \mid \mathbf{x}_i; \mathbf{W}, \mathbf{b}) \Big) \; + \; \frac{\lambda}{2} (\|\mathbf{W}\|^2 + \|\mathbf{b}\|^2) \tag{5.14}$$

To perform a search for the cost function's minimum, the Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) algorithm (Liu; Nocedal, 1989) is employed. L-BFGS is a quasi-Newton optimization algorithm that builds an efficient low-memory approximation of the inverse Hessian, enabling faster convergence than first-order (gradient based) methods while remaining viable for large-scale problems. In this case study, the `scikit-learn` 1.4.2 (Pedregosa et al., 2011) implementation of multinomial logistic regression with the L-BFGS solver was used.

Having fully defined the liquid state machine architecture, its parameters are tuned for each of the datasets chosen for this study: Spiking Heidelberg Digits and N-MNIST.

5.3.2 Spiking Heidelberg Digits Dataset

In order to tune a reservoir to the SHD benchmark, initially the network connectivity parameters were set to the same values adopted by Biswas et al. (2024). The network is structured as a three-dimensional grid of size $(10, 10, 10)$, yielding 1000 reservoir neurons in total. Later, these parameters were hand-tuned seeking to optimize the final accuracies obtained with the reservoir, with tests being guided by the changes in network

dynamics. Neuron parameters were defined according to Table 5, setting values similar to biological neurons (Gerstner; Kistler, 2002).

Table 5 – LIF neuron parameters for the SHD benchmark.

Parameter	**Symbol**	**Value**
Membrane time constant	τ_m	100 ms
Membrane capacitance	C_m	200 pF
Resting potential	v_{rest}	-70 mV
Reset potential after spike	v_{reset}	-70 mV
Spike threshold	v_{thresh}	-50 mV
Refractory period	$t_{\text{refractory}}$	2 ms

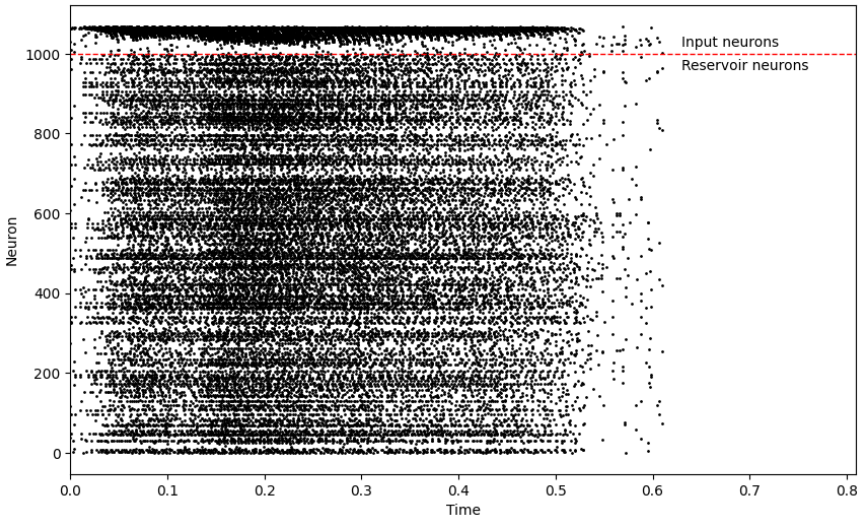
The network connection parameters are listed in Table 6. Excitatory synaptic weights are uniformly sampled from the interval $[0.4, 1.2]$ pC, and inhibitory weights from the interval $[-2.4, -0.8]$ pC. The synaptic delays are defined based on the Euclidean distances between connected nodes, such that the delay for a synapse between neurons at positions $i, j \in \mathbb{N}_0^3$ is $K_d \|x_i - x_j\|_2$, with $K_d = 5\,\text{ms}$.

Table 6 – Connection parameters for the SHD benchmark

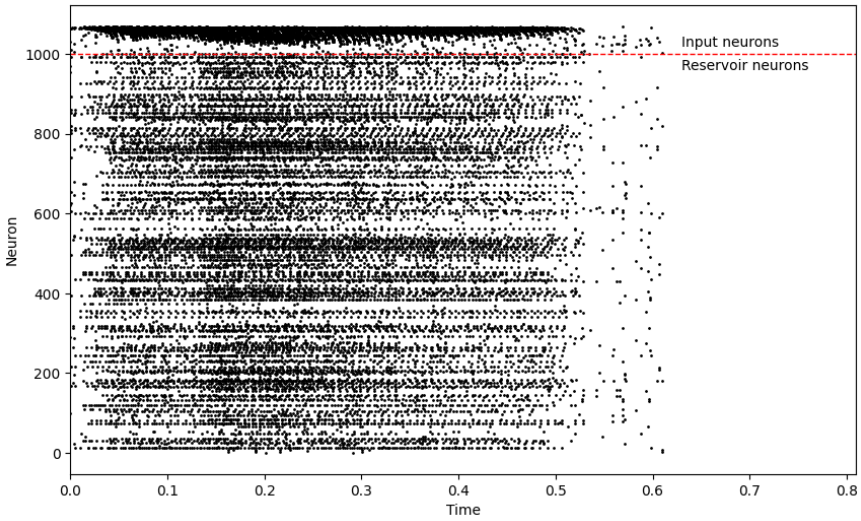
Parameter	**Value**
λ	2.0
C_{EE}	0.3
C_{EI}	0.2
C_{IE}	0.4
C_{II}	0.0

The input layer is defined with $F_{out} = 5$, that is, each spiking input channel has a fan-out of 5 synapses connecting it to reservoir LIF neurons. A high constant weight of +6 pC (+30 mV excursion with the chosen membrane capacitance) is set for all input synapses. The associated synaptic delays are set to a constant value of 0.1 ms.

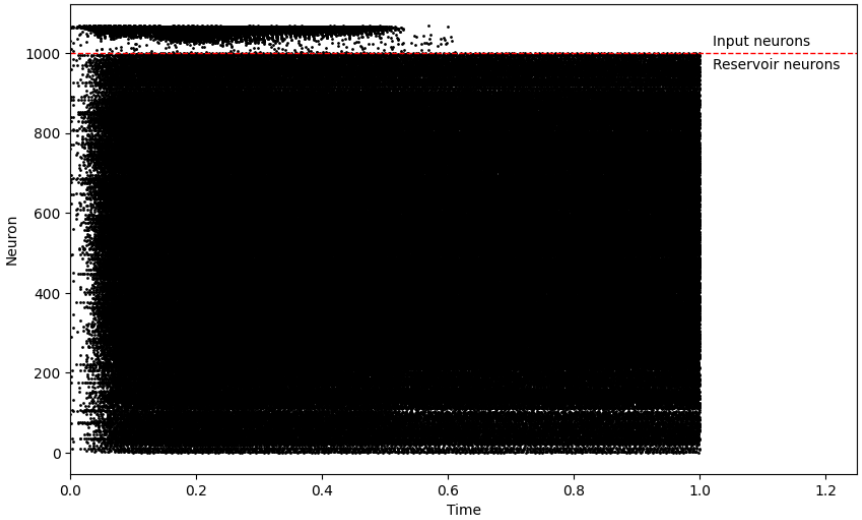
The hand-tuning process that led to these parameter choices was guided by two heuristics: simplicity and balanced network dynamics. Firstly, the parameters are either set to constants or sampled from simple distributions, with magnitudes similar to the ones found in literature (Biswas et al., 2024). Secondly, the network is tuned to provide a sufficient level of asynchronous spiking activity, avoiding both high-inhibition and low-inhibition regimes, which tend to lead to poor LSM performance. Figure 20 depicts the reservoir spiking activity produced by an SHD audio sample for both the ideal and non-ideal activity regimes. In the ideal regime, the activity is driven by both inputs and recurrent connections, establishing memory capacity as well as nonlinear processing (Maass; Natschläger; Markram, 2002). In contrast, in the high-inhibition regime, activity is primarily input-driven, failing to provide these desired properties. Lastly, in the low-inhibition regime, activity becomes saturated and dominated by recurrent connections, thereby failing to capture the influence of the inputs.



(a) Ideal regime



(b) High-inhibition regime: activity is mainly driven by inputs.



(c) Low-inhibition regime: activity is self-sustained.

Figure 20 – Spiking activity for an SHD audio sample under ideal and non-ideal network activity regimes (simulation duration of 1 second).

All training and test samples are run through the simulator as a continuous stream, with an additional margin of 500 ms to allow for the separation between inputs. This margin is sufficient for the neuron's exponential decay to bring its potential back near the resting state, thereby minimizing interference between distinct inputs.

Firstly, the WTA classifier method is tested. The discretization time step is set to 50 ms, with a kernel decay factor of 300 ms. Both parameters were empirically tuned to achieve the best results given the available memory. The multinomial logistic regression model is trained on the design matrix and the associated labels for each time step. Once trained, the model is used to produce the corresponding predictions. Then, these predictions are aggregated into predictions for each digit utterance using the WTA method. This approach yields a training accuracy of 93.10% and a test accuracy of 71.33%. The corresponding confusion matrix for the test set is shown in Figure 21.

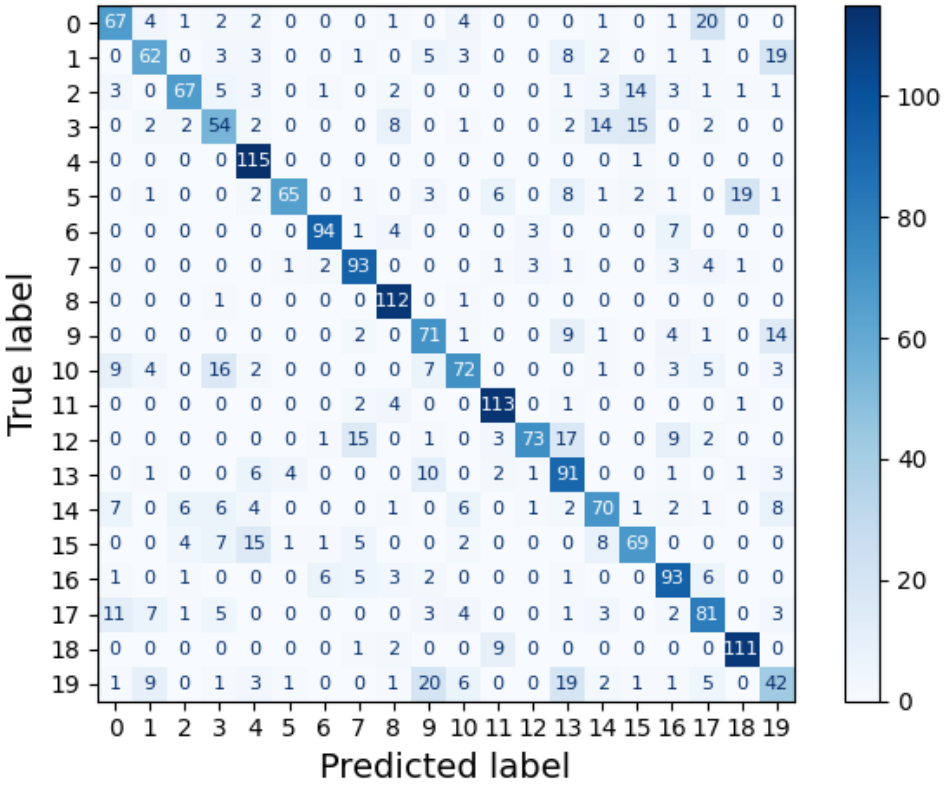


Figure 21 – Confusion matrix on the SHD test set for the WTA readout method.

Secondly, the readout method based on reservoir spike counts for each audio sample is used to train the classifier, yielding a training accuracy of 94.56% and a test accuracy of 78.09%. The corresponding test-set confusion matrix is shown in Figure 22. The spike-count classifier outperforms the WTA classifier, a result that may stem from the fact that spike counts can indirectly convey information about the input duration. Nevertheless, both have the same number of parameters, as both methods use feature vectors with size equal to the number of LSM neurons (1000). Since the softmax regression is defined over 20 classes, 20020 parameters (20000 weights and 20 biases) are used.

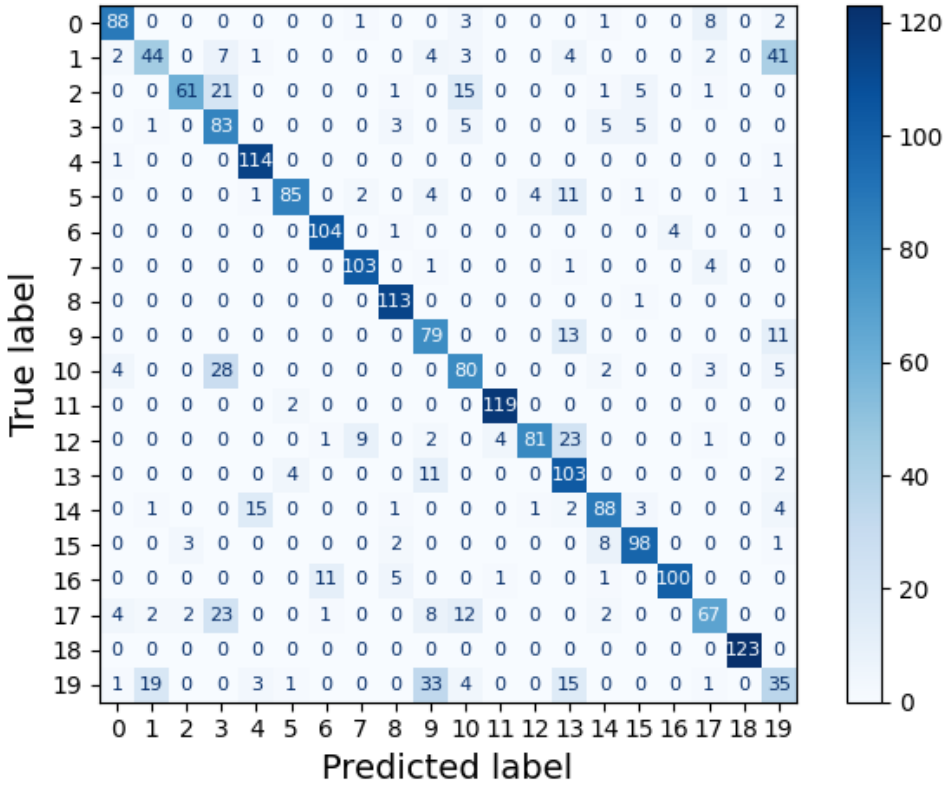


Figure 22 – Confusion matrix on the SHD test set for the spike count readout method.

A regularization factor of $\lambda = 100$ is used for both models, obtained though a linear search aimed at optimizing test accuracy. Although the gap between the training and test accuracies indicates overfitting, adjusting the regularization factor does not lead to accuracy improvements. Further strategies have been employed in the literature to reduce overfitting, such as adding noise to the input data (Cramer et al., 2022), which may represent a promising direction for future work to improve accuracy. Nevertheless, the results still demonstrate successful classification on the test set, comparable to state-of-the-art LSMs.

Table 7 provides a comparison between the proposed models and the state of the art for the SHD dataset. It is noticeable that the spike-count classifier yields an accuracy comparable to the state-of-the-art LSM. Nevertheless, it is still limited by the gap between the best performing LSMs and the best performing models in general.

In addition to the classification results, running all 10420 samples through the network, with additional overhead from computing and storing the feature vectors for both the WTA and spike-count methods, took a total of 983 seconds. This corresponds to an average processing time of 94.3 ms per sample.

The total duration of all audio files in the SHD dataset is 7513 seconds. Consequently, the system requires an average of 130.8 ms to simulate 1 second of audio. Therefore, the performance requirement NFR1 contained in Table 3 is validated, as the simulation proceeds faster than the corresponding audio duration.

Table 7 – Classification accuracies and parameter counts of the best-performing models on the SHD benchmark, including the models proposed in this work.

Model	**Test accuracy (%)**	**Trainable parameters**
Sun et al. (2025)	96.26 ± 0.08	200 000
Schöne et al. (2024)	95.90	400 000
Baronig et al. (2025)	95.81 ± 0.56	450 000
Hammouamri, Khalfaoui-Hassani and Masquelier (2023)	95.07 ± 0.24	200 000
	...	
Ours (Spike-count classifier)*	78.09	20 020
Biswas et al. (2024)*	77.80	30 000
Ours (WTA classifier)*	71.33	20 020
Krenzer and Bogdan (2025)*	65.80	–

* Liquid state machine approaches.

5.3.3 N-MNIST Dataset

In order to evaluate the potential generalization property of liquid state machines to different tasks, the same reservoir architecture, connection parameters and neuron parameters used for the SHD dataset are adopted for the N-MNIST benchmark. Only the input layer is altered, by reducing the input fan-out to $F_{out} = 2$ and decreasing synaptic weights to +2 pC (+10 mV excursion with the chosen membrane capacitance), adapting it to an increased number of input channels. The aim is to prevent the network dynamics from being dominated by input spikes, requiring a reduction in fan-out to limit the number of input-driven LSM neurons.

The N-MNIST dataset is composed of events organized in a 34×34 spatial grid, with two polarities (ON and OFF), yielding a total of 2312 input channels. In order to reduce input dimensionality, the events are firstly flattened into 2312 channels and then grouped into sets of 10 adjacent channels, resulting in 232 input aggregated channels. This compact representation is used as the input to the network's input layer. This approach enables the use of the defined input layer, allowing the application of the same method used for SHD for comparison purposes. Nevertheless, input layers that exploit the two-dimensional input structure may yield better results, representing a promising direction for future exploration.

With this configuration, all available samples are run through the simulator as a continuous stream, with a margin of 500 ms between samples to facilitate the separation of inputs. This margin is sufficient for the neuron membrane potentials to return to their resting states, thereby minimizing interference between samples.

Firstly, the WTA classification approach is tested. The discretization time step is set to 300 ms, constrained by memory limitations of the available hardware. For the simulated time steps, a multinomial logistic regression is fit on the attained design matrix

and associated labels. Then, the aggregated WTA predictions are calculated, yielding a training accuracy of 96.05% and test accuracy of 94.77%. The corresponding test-set confusion matrix is detailed in Figure 23.

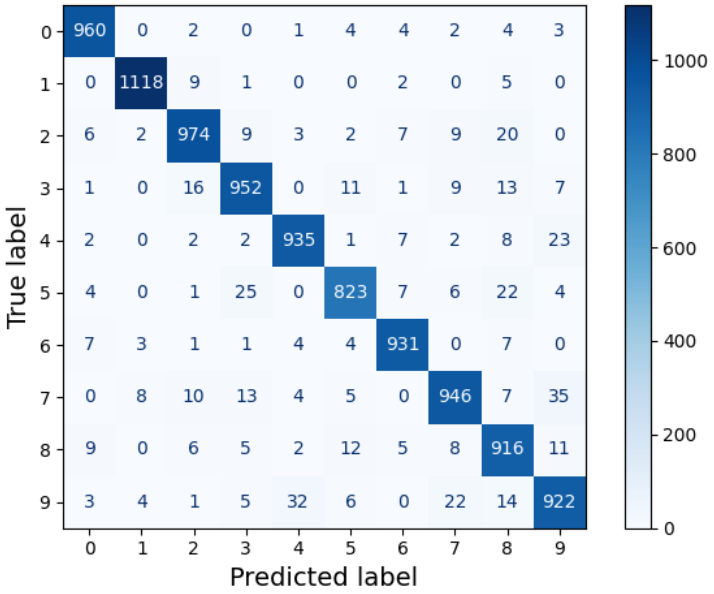


Figure 23 – Confusion matrix on the N-MNIST test set for the WTA readout method.

Secondly, the same experimental data is used to train a readout layer based on the spike counts over full dataset samples, yielding a training accuracy of 96.78% and test accuracy of 95.16%. The corresponding confusion matrix for the test set is shown in Figure 24. Both the WTA and spike-count methods use the same number of trainable parameters, since the feature vector size is equal to the number of LSM neurons. Thus, the softmax classifier for 10 classes contains a total of 10010 parameters.

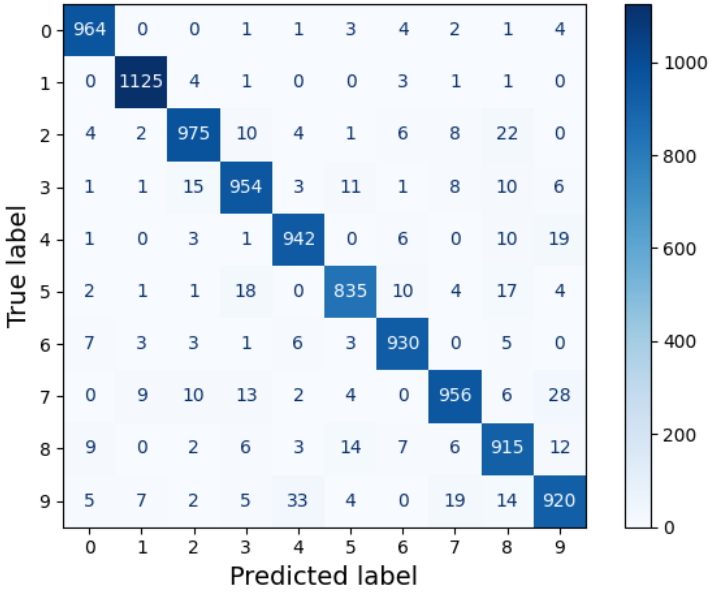


Figure 24 – Confusion matrix on the N-MNIST test set for the spike count readout method.

The achieved accuracy falls below the state-of-the-art LSM architecture by Biswas

et al. (2024), which achieved an accuracy of 98.1%. Nevertheless, this result remains reasonable given the significantly smaller network size used in this project and the absence of ensemble methods. Table 8 presents a comparison between the achieved performance and the architectures from the literature examined in this work. This comparison is not intended to be exhaustive with respect to all known LSM and SNN architectures.

Table 8 – Classification accuracies and parameter counts of the best-performing models on the N-MNIST benchmark, including the models proposed in this work.

Model	**Test accuracy (%)**	**Trainable parameters**
Zhang et al. (2024)	99.57 ± 0.10	–
Biswas et al. (2024)*	98.10	36 000
Ours (Spike-count classifier)*	95.16	10 010
Ours (WTA classifier)*	94.77	10 010
Orchard et al. (2015)	83.44	–

* Liquid state machine approaches.

In addition to the classification results, the total execution time for the 70000 files in the N-MNIST dataset was 1939 seconds, including the overhead associated with feature processing and storage. This corresponds to an average of 27.7 ms per file. Since each N-MNIST file contains approximately 300 ms of recorded data, the system requires, on average, 92.3 ms to process 1 second of N-MNIST data. This further validates the performance requirement NFR1, listed in Table 3.

6 Discussion

This chapter provides a critical reflection on the results presented in Chapter 5, examining the implementation's advantages and drawbacks, its performance in comparison with existing simulators, and its applicability to machine learning tasks under the LSM framework. In addition, possible directions for further development are also discussed.

6.1 Simulator Implementation

The developed spiking neural network simulator distinguishes itself from the literature due to its purely event-based simulation paradigm. This approach offers advantages in terms of computational efficiency in certain cases, particularly in scenarios with sparse spiking activity. Nevertheless, this choice also imposes limitations on the neuron models that can be implemented. Thus, the simulator is currently restricted to neuron models compatible with event-based updates, such as the LIF model.

In sum, although this design choice restricts the range of neuron models compatible with the reservoir, it can yield efficiency gains in several scenarios. Moreover, it renders the implementation particularly compact and enables fast network setup times. This minimal structure, combined with its compatibility with single-threaded execution, due to its inherently sequential simulation procedure, and its availability as a *C++* library, makes it a reasonable option for embedded systems.

6.2 Performance and Correctness

Using the developed simulator, comparative experiments show a satisfactory performance level, being comparable to existing simulators in the literature, such as Brian 2 (Stimberg; Brette; Goodman, 2019) and NEST (Diesmann; Gewaltig, 2001). Nevertheless, a scaling analysis shows there is still room for improvement, as the execution time scales more sharply with the network size than other simulators. A variety of changes can be formulated to improve the scaling property, taking inspiration from strategies already explored in literature.

The dynamics observed in the comparative study are consistent across simulators, within the constraints imposed by chaotic behavior, and align with established dynamic regimes for the neuron models considered. This evidence, together with extensive unit tests, attests to the correctness of the implementation and supports its use for simulating spiking neural networks.

Hence, the results obtained for this initial dynamics assessment validate the exploration of this simulator in machine learning tasks under the LSM framework. Concurrently, it highlights that performance optimizations and alternative neuron models can also be explored in order to bridge the performance gap for large-scale networks between this simulator and its well-established alternatives.

6.3 Applicability to Machine Learning Tasks

After validating the simulator's correctness, experiments performed with the proposed LSM architecture for the SHD benchmark highlight the simulator's applicability to machine learning problems and provide further proof of its correct implementation. Although a simple reservoir topology and neuron model are adopted, the attained accuracies for this audio classification task show evidence of this architecture's capacity for processing time-dependent inputs, reaching accuracies similar to the current state-of-the-art LSM model. Nevertheless, the discrepancy between training and test accuracies indicates overfitting, motivating further investigation of its causes and the application of methods to mitigate it. This leaves room for improvement by enhancing the model's generalization capabilities.

The same study was conducted on the N-MNIST benchmark, providing additional evidence of the simulator's applicability to solving machine learning tasks. Although state-of-the-art level performance was not achieved, results surpass early solutions by a large margin. To further improve accuracy, multiple approaches could be taken, such as additional network parameter tuning, increasing the network size, or applying ensemble methods. Thus, there remains a substantial margin for improvement. Nevertheless, the current solution is sufficient to support its efficiency, particularly given the simple architecture adopted in this work.

For both benchmarks, further performance improvements should also possible by taking inspiration from more advanced bio-inspired neural network models. For example, STDP could be incorporated into the synapse model to introduce unsupervised learning into the network's topology (Song; Miller; Abbott, 2000). Furthermore, alternative reservoir and input-layer topologies could be explored by applying different connection rules. Lastly, the set of hyperparameters that define the LSM could be tuned with gradient-free optimization algorithms, such as grid search, random search or Bayesian optimization.

6.4 Requirements Verification

Based on the obtained results, an assessment was conducted to evaluate whether the functional and non-functional requirements were met. Table 9 summarizes the functional

requirements and their verification status, while Table 10 presents the non-functional requirements and their corresponding status.

Regarding the functional requirements, neuron and network simulation capabilities (FR1, FR2, FR3, and FR4), necessary for modeling LSMs, were validated through the simulator's behavioral assessment and its comparison with established solutions. Low-level control (FR5) is ensured by the extensible object-oriented design of the *C++* simulation kernel. Finally, the *Python* interface, which directly interfaces with the kernel, validates the accessible interface requirement (FR6).

Table 9 – Functional requirements for the simulator and their validation status

ID	Requirement	Validated
FR1	**Neuron simulation:** The simulator shall implement mathematical models for neurons, reproducing behavior observed in biological counterparts.	Yes
FR2	**Network simulation:** The simulator shall allow the combination of neuron objects into networks and support their simulation under external stimuli.	Yes
FR3	**Event-driven computation:** The simulator shall process information through discrete events rather than continuous activations used in conventional ANNs.	Yes
FR4	**Experiment execution:** The simulator shall support experiments with access to spiking activity and network states throughout execution, enabling neural behavior studies and machine learning applications.	Yes
FR5	**Low-level control:** The simulator shall provide direct control over computational procedures, supporting the extension of existing numerical representations and processing strategies through the creation of subclasses.	Yes
FR6	**Accessible interface:** A high-level programmatic interface shall expose the simulator kernel, enabling the definition of simulations, experiment execution, and result retrieval. Simulation results shall be retrievable as *NumPy* arrays.	Yes

The non-functional requirements were also met. The performance level (NFR1) is consistent with existing simulators and satisfies the defined execution-time baseline by a substantial margin in the studied cases, highlighting its potential for simulating larger-scale networks. Scalability requirements (NFR2) were likewise fulfilled, with both the single-neuron and network tests executed successfully, featuring randomly connected networks composed of 1000 neurons with a connection density of 10%. Maintainability and adaptability (NFR3) are supported by the extensible object-oriented software design. Finally, the accessible interface requirement (NFR4) is met through the user-friendly high-level Python interface and its compatibility with standard analysis tools. This is achieved by adopting classes with interfaces similar to those of Brian2 (Stimberg; Brette; Goodman, 2019) and NEST (Diesmann; Gewaltig, 2001), such as in the definition of

monitors (`SpikeMonitor` and `StateMonitor`) and the execution function (`run`). User testing could be conducted in the future for further verification and to refine the interface.

Table 10 – Non-functional requirements for the simulator and their validation status

ID	Requirement	Validated
NFR1	**Performance:** The simulator shall be computationally efficient, achieving competitive performance. As an acceptance criterion, a liquid state machine of 1,000 neurons must simulate 1 second of biological time in under 1 second of elapsed time. Benchmark architectures similar to those used by Stimberg, Brette and Goodman (2019), who introduced Brian 2, will also be used for comparison, with the simulator expected to perform comparably to Brian 2 and NEST.	Yes
NFR2	**Scalability:** The simulator shall support networks of varying sizes. As a baseline, it must simulate both a single-neuron network and a 1,000-neuron network with 10% connection density (100,000 synapses).	Yes
NFR3	**Adaptability:** The codebase shall be designed for easy extension and modification of existing functionalities, using object-oriented programming to create generic classes from which multiple implementations can inherit.	Yes
NFR4	**Accessible interface:** The programmatic interface shall be intuitive, with the use of clear naming. It shall also be user-friendly, which should be validated with user testing. If user testing is not feasible, this requirement may be preliminarily validated through an analysis of the interface design.	Yes

In conclusion, all requirements were verified, supporting the successful execution of the project.

6.5 Summary and Perspectives

Ultimately, the simulator exhibits the expected behavior, satisfying the requirements defined at the beginning of the project. Moreover, its performance is competitive, proving sufficiently efficient to be applied to commonly used datasets within a reasonable time frame. The experiments conducted on the proposed LSM architecture demonstrate that it can effectively address machine learning tasks, confirming its applicability to the creation of practical solutions.

Directions for future work include exploring modifications to the simulation algorithm to improve performance for large-scale networks and enable parallelism, as well as assessing refinements to the reservoir computing approach, either through optimization of the current architecture or through the inclusion of plasticity mechanisms and additional neural models to the simulator.

Another possibility is to use the developed simulator as a foundation for high-level synthesis, enabling software testing of networks followed by their synthesis into neuromorphic hardware. In an initial phase, this could target the hardware architecture designed by the author as part of a double-degree program in France, which motivated this project, with the potential for later extension to other existing neuromorphic architectures.

7 Conclusion

The work herein developed set out to design, implement and evaluate a neural activity simulator, focused on spiking neural networks, capable of supporting machine learning tasks within the liquid state machine framework.

In this document, the design choices and architecture for this spiking neural network simulator are detailed, along with the theoretical context used to build this solution. From this foundation, a simulation kernel accompanied by a user-friendly interface were developed and documented. With this implementation, experiments were performed to test both the simulator dynamics test and its applicability to machine learning tasks.

Initially, the context surrounding the work was researched, providing the foundation for defining the motivations for creating a novel simulator of neural activity. This phase served to guide the definition of objectives and an implementation plan. From this research, the theoretical background relevant to the project was also consolidated, providing information on the broader context in which it is situated and leading to the definition of the mathematical models used throughout it.

After establishing this foundation, the functional and non-functional requirements were formally defined. From these requirements, the methods to be employed in the project's development were derived, and the technologies to be used, such as programming languages and their respective libraries, were identified. Finally, both the overall structure of the event-driven simulator and the detailed implementation's architecture fully designed.

With the simulator implemented, experiments were conducted to verify the system's behavior and to perform a comparative evaluation of its performance against well-established simulators. The results demonstrate that the simulator is compatible with existing solutions and reproduces the behavior reported in studies of the dynamical system, while also exhibiting competitive performance. These results showed that the simulator satisfied the functional requirements proposed at the start of the project.

Having validated the simulators overall dynamics, the SHD benchmark audio classification task (Cramer et al., 2022) was solved with a bio-inspired LSM architecture instantiated on the simulator. The attained accuracies are comparable to those of the current state-of-the-art LSM models, demonstrating both the effectiveness of the proposed method and the applicability of the simulator to such scenarios.

Then, another machine learning case study was developed within the LSM framework by addressing the N-MNIST (Orchard et al., 2015) image classification task, providing further insight into practical applications of the simulator. Attained accuracies demonstrate

a good level of performance compared to the literature, providing additional evidence of the proposed architecture's effectiveness.

Together, these steps demonstrate not only the simulator's correctness and efficiency, but also its potential as a foundation for future research and applications in neuroscience-inspired computation.

Therefore, in conclusion, the development of the spiking neural network simulator was successful, having been validated by set of experiments devised to show that the current implementation presents consistent behavior and competitive performance relative to the state of the art. Furthermore, its application to machine learning tasks shows promising results, as levels of performance matching state-of-the-art models were obtained for the SHD task. Hence, both the functional and non-functional requirements were met, confirming the successful completion of the project.

Possible directions for future work include extending the simulator's functionalities and refining the developed case studies. Extensions may involve adding synaptic plasticity mechanisms and incorporating additional neuron models. Other improvements would focus on modifying the execution procedure to enhance performance on large-scale networks and enable parallel execution. In addition to improvements to the simulator, the adopted LSM approach could be refined to achieve better performance, either through hyperparameter optimization or architectural changes. Lastly, the developed simulator could serve as a foundation for high-level synthesis, enabling software testing prior to synthesizing networks into neuromorphic hardware.

References

ANTONIK, P. et al. Human action recognition with a large-scale brain-inspired photonic computer. *Nature Machine Intelligence*, v. 1, n. 11, p. 530–537, 2019. ISSN 2522-5839.

BARONIG, M. et al. Advancing spatio-temporal processing through adaptation in spiking neural networks. *Nature Communications*, v. 16, n. 1, p. 5776, jul. 2025. ISSN 2041-1723.

BISWAS, A. et al. Temporal and Spatial Reservoir Ensembling Techniques for Liquid State Machines. In: *2024 International Conference on Neuromorphic Systems (ICONS)*. Los Alamitos, CA, USA: IEEE Computer Society, 2024. p. 78–81.

BRUNEL, N. Dynamics of sparsely connected networks of excitatory and inhibitory spiking neurons. *J. Comput. Neurosci.*, v. 8, n. 3, p. 183–208, may 2000.

CAI, H. et al. Brain organoid reservoir computing for artificial intelligence. *Nature Electronics*, v. 6, n. 12, p. 1032–1039, 2023. ISSN 2520-1131.

CHUNDI, P. K. et al. Always-On Sub-Microwatt Spiking Neural Network Based on Spike-Driven Clock- and Power-Gating for an Ultra-Low-Power Intelligent Device. *Frontiers in Neuroscience*, v. 15, p. 684113, jul. 2021. ISSN 1662-453X.

CRAMER, B. et al. The Heidelberg Spiking Data Sets for the Systematic Evaluation of Spiking Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems*, v. 33, n. 7, p. 2744–2757, jul. 2022. ISSN 2162-237X, 2162-2388.

D'AGOSTINO, S. et al. DenRAM: Neuromorphic dendritic architecture with RRAM for efficient temporal processing with delays. *Nature Communications*, v. 15, n. 1, p. 3446, apr. 2024. ISSN 2041-1723.

DIESMANN, M.; GEWALTIG, M.-O. Nest: An environment for neural systems simulations. *Forschung und wisschenschaftliches Rechnen, Beiträge zum Heinz-Billing-Preis*, v. 58, p. 43–70, 2001.

DU, C. et al. Reservoir computing using dynamic memristors for temporal information processing. *Nature Communications*, v. 8, n. 1, p. 2204, dec. 2017. ISSN 2041-1723.

FAN, X.; MARKRAM, H. A brief history of simulation neuroscience. *Frontiers in Neuroinformatics*, Volume 13 - 2019, 2019. ISSN 1662-5196.

GERSTNER, W.; KISTLER, W. M. *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge, United Kingdom: Cambridge University Press, 2002.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge, Massachusetts: MIT Press, 2016.

HAMMOUAMRI, I.; KHALFAOUI-HASSANI, I.; MASQUELIER, T. *Learning Delays in Spiking Neural Networks using Dilated Convolutions with Learnable Spacings*. 2023. Available at: https://arxiv.org/abs/2306.17670. Accessed: 09 Dec. 2025.

HARRIS, C. R. et al. Array programming with NumPy. *Nature*, Springer Science and Business Media LLC, v. 585, n. 7825, p. 357–362, sep. 2020.

HINES, M. L.; CARNEVALE, N. T. The neuron simulation environment. *Neural Computation*, v. 9, n. 6, p. 1179–1209, 08 1997. ISSN 0899-7667.

HODGKIN, A. L.; HUXLEY, A. F. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of physiology*, v. 117, n. 4, p. 500, 1952.

ISAACSON, J. S.; SCANZIANI, M. How Inhibition Shapes Cortical Activity. *Neuron*, v. 72, n. 2, p. 231–243, oct. 2011. ISSN 08966273.

JAEGER, H. The "echo state" approach to analysing and training recurrent neural networks-with an erratum note. *Bonn, Germany: German national research center for information technology gmd technical report*, Bonn, v. 148, n. 34, p. 13, 2001.

KRENZER, D.; BOGDAN, M. Reinforced liquid state machines—new training strategies for spiking neural networks based on reinforcements. *Frontiers in Computational Neuroscience*, v. 19, p. 1569374, may 2025. ISSN 1662-5188.

LAPICQUE, L. Quantitative investigations of electrical nerve excitation treated as polarization. 1907. *Biological Cybernetics*, Springer, v. 97, n. 5-6, p. 341–349, 2007. ISSN 0340-1200.

LARGER, L. et al. High-speed photonic reservoir computing using a time-delay-based architecture: Million words per second classification. *Phys. Rev. X*, American Physical Society, v. 7, p. 011015, Feb 2017.

LECUN, Y. et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, v. 86, n. 11, p. 2278–2324, 1998.

LI, J. et al. Bio-inspired intelligence with applications to robotics: A survey. *Intelligence & Robotics*, 2021.

LIU, D. C.; NOCEDAL, J. On the limited memory bfgs method for large scale optimization. *Mathematical Programming*, v. 45, n. 1, p. 503–528, Aug 1989. ISSN 1436-4646.

LUKOšEVIčIUS, M.; JAEGER, H. Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, v. 3, n. 3, p. 127–149, 2009. ISSN 1574-0137.

MAASS, W. Networks of spiking neurons: The third generation of neural network models. *Neural Networks*, v. 10, n. 9, p. 1659–1671, 1997. ISSN 0893-6080.

MAASS, W.; NATSCHLÄGER, T.; MARKRAM, H. Real-Time Computing Without Stable States: A New Framework for Neural Computation Based on Perturbations. *Neural Computation*, v. 14, n. 11, p. 2531–2560, nov. 2002. ISSN 0899-7667, 1530-888X.

MATTIA, M.; GIUDICE, P. D. Efficient event-driven simulation of large networks of spiking neurons and dynamical synapses. *Neural Computation*, v. 12, n. 10, p. 2305–2329, 2000.

MEINECKE, D. L.; PETERS, A. GABA immunoreactive neurons in rat visual cortex. *Journal of Comparative Neurology*, v. 261, n. 3, p. 388–404, jul. 1987. ISSN 0021-9967, 1096-9861.

NEFTCI, E. O.; MOSTAFA, H.; ZENKE, F. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, v. 36, n. 6, p. 51–63, 2019.

ORCHARD, G. et al. Converting Static Image Datasets to Spiking Neuromorphic Datasets Using Saccades. *Frontiers in Neuroscience*, v. 9, nov. 2015. ISSN 1662-453X.

PEDREGOSA, F. et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.

ROSENBLATT, F. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, American Psychological Association, v. 65, n. 6, p. 386, 1958.

SANDE, G. V. der; BRUNNER, D.; SORIANO, M. C. Advances in photonic reservoir computing. *Nanophotonics*, v. 6, n. 3, p. 561–576, 2017.

SCHÖNE, M. et al. Scalable event-by-event processing of neuromorphic sensory signals with deep state-space models. In: *2024 International Conference on Neuromorphic Systems (ICONS)*. Arlington, Virginia, USA: IEEE, 2024. p. 124–131.

SONG, S.; MILLER, K. D.; ABBOTT, L. F. Competitive hebbian learning through spike-timing-dependent synaptic plasticity. *Nature Neuroscience*, v. 3, n. 9, p. 919–926, sep. 2000. ISSN 1546-1726.

STIMBERG, M.; BRETTE, R.; GOODMAN, D. F. Brian 2, an intuitive and efficient neural simulator. *eLife*, eLife Sciences Publications, Ltd, v. 8, p. e47314, aug 2019. ISSN 2050-084X.

SUMI, T. et al. Biological neurons act as generalization filters in reservoir computing. *Proceedings of the National Academy of Sciences*, v. 120, n. 25, p. e2217008120, 2023.

SUN, P. et al. Towards parameter-free attentional spiking neural networks. *Neural Networks*, v. 185, p. 107154, may 2025. ISSN 08936080.

TONIN, L.; MILLÁN, J. D. R. Noninvasive Brain–Machine Interfaces for Robotic Devices. *Annual Review of Control, Robotics, and Autonomous Systems*, v. 4, n. 1, p. 191–214, may 2021. ISSN 2573-5144, 2573-5144.

VREESWIJK, C. van; SOMPOLINSKY, H. Chaos in neuronal networks with balanced excitatory and inhibitory activity. *Science*, v. 274, n. 5293, p. 1724–1726, 1996.

WIJESINGHE, P. et al. Analysis of Liquid Ensembles for Enhancing the Performance and Accuracy of Liquid State Machines. *Frontiers in Neuroscience*, v. 13, p. 504, may 2019. ISSN 1662-453X.

YAMAZAKI, K. et al. Spiking Neural Networks and Their Applications: A Review. *Brain Sciences*, v. 12, n. 7, p. 863, jun. 2022. ISSN 2076-3425.

YAN, M. et al. Emerging opportunities and challenges for the future of reservoir computing. *Nature Communications*, v. 15, n. 1, p. 2056, mar. 2024. ISSN 2041-1723.

ZHANG, S.-Q. et al. On the intrinsic structures of spiking neural networks. *J. Mach. Learn. Res.*, JMLR.org, v. 25, n. 1, jan. 2024. ISSN 1532-4435.

ZHONG, Y. et al. A memristor-based analogue reservoir computing system for real-time and power-efficient signal processing. *Nature Electronics*, v. 5, n. 10, p. 672–681, 2022. ISSN 2520-1131.